2024

PREMIER PAS EN PHP

godevijunior@gmail.com
ESAG-NDE

Sommaire

1. **PHP tutorial : ce que vous devez prévoir**
2. **Bases de PHP : la syntaxe du langage de script**
3. **Les superglobals $_GET et $_POST**
4. **L'élément if et les opérateurs comparatifs PHP**
5. **Les boucles (while, for)**
6. **Les opérations sur des fichiers**

PHP tutorial : ce que vous devez prévoir

Notre tutoriel s'adresse avant tout aux débutants. Afin de mieux comprendre la plupart des exemples, il est cependant utile de connaître les bases du développement Web moderne et du HTML. Afin de suivre les exemples présentés sur votre ordinateur, les prérequis suivants doivent être respectés :

- Serveur Web incluant un **interprète PHP**
- PHP installé
- Navigateur Web
- Éditeur de texte

Pour le serveur, nous vous recommandons d'utiliser l'environnement de test local XAMPP, qui est fourni gratuitement par Apache Friends pour les systèmes d'exploitation Windows, Linux et macOS. **XAMPP est purement un serveur de test.** Le pack logiciel permet aux développeurs Web de mettre en place très rapidement un environnement de test complet pour les scripts, les pages HTML et les feuilles de style. Un fonctionnement sécurisé sur Internet en tant que serveur Web n'est **pas garanti** pour autant. Un guide d'installation détaillé est disponible dans notre tutoriel XAMPP.

Hébergement Web avec conseiller personnel !

Hébergement Web puissant, flexible et performant avec boîte email, conseiller personnel et domaine inclus la 1ère année !

Domaine

Certificat SSL

Assistance 24/7

Continuer

Bases de PHP : la syntaxe du langage de script

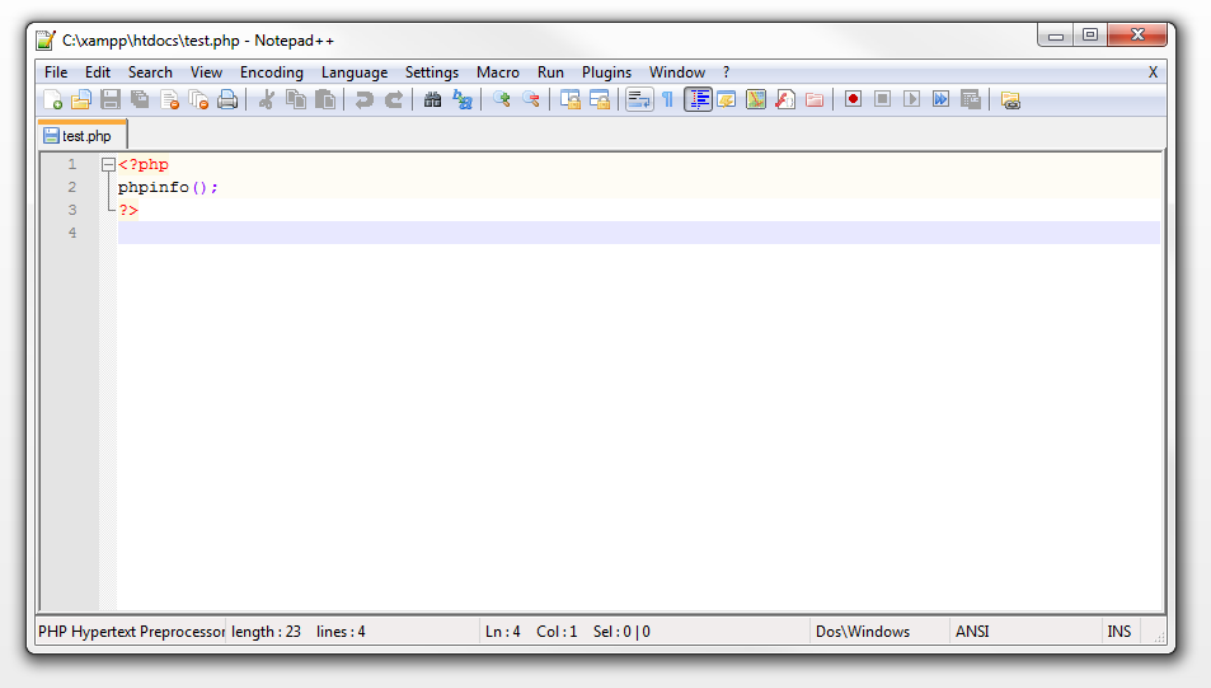
Si vous avez configuré votre serveur Web local (par exemple en utilisant XAMPP), vous devez vérifier si PHP a été installé correctement et est prêt à exécuter des scripts. Un script est un programme qui n'est pas préalablement compilé en code binaire. Il est en effet exécuté par un **interprète**. Ouvrez votre **éditeur de texte** favori et importez le script PHP suivant :

<?php

phpinfo();

?>

php



Un éditeur de texte tel que Notepad++ vous aide à programmer grâce à la coloration syntaxique.

Les scripts PHP présentent toujours une structure au schéma identique. La balise PHP ouvrante <?php indique qu'un **environnement script** est débuté. Elle est suivie du code PHP, présenté sous forme d'instructions. Dans notre exemple, la fonction *phpinfo()* est appelée. La plupart des fonctions PHP nécessitent un ou plusieurs paramètres entre parenthèses ; dans *phpinfo()*, ces paramètres sont optionnels : *phpinfo(INFO_ALL)*. Chaque énoncé se termine par un point-virgule (*;*). Enfin, pour fermer l'environnement script, on utilise la balise PHP fermante suivante : ?>.

Definition

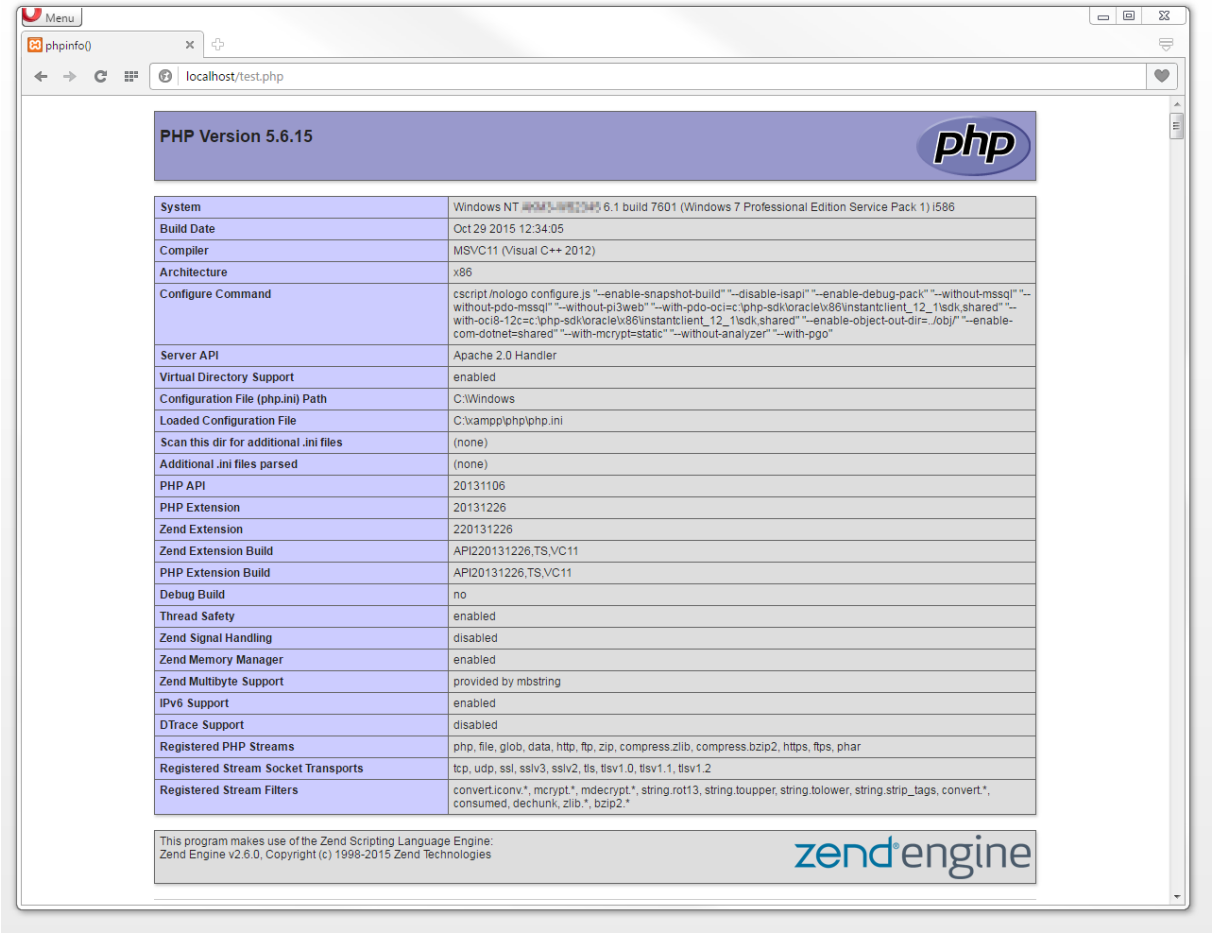
Les fonctions sont des sous-catégories permettant d'externaliser des parties du code du programme. Pour éviter la redondance, les tâches récurrentes peuvent être définies la première fois en tant que fonction, puis appelées à l'aide d'un nom de fonction. Les développeurs Web utilisent pour ce faire des fonctions PHP prédéfinies ou créent leurs propres sous-programmes.

Sauvegardez le fichier texte au format *.php* (script PHP) en le renommant *test*, puis démarrez votre serveur Web. Si vous utilisez l'environnement de test XAMPP, placez *test.php* dans le répertoire XAMPP à l'emplacement *htdocs* (*C:\xampp\htdocs*).

Le fichier d'exemple est ensuite accessible via l'URL suivante dans le navigateur Web : http://localhost/test.php. Si vous utilisez un serveur Web différent ou une configuration personnalisée du logiciel XAMPP, **sélectionnez l'URL en fonction du chemin de fichier correspondant**.

Saisissez l'URL http://localhost/test.php pour ordonner à votre navigateur Web de demander le fichier *test.php* au serveur Web. Le serveur Apache HTTP Server ou le logiciel de serveur Web de votre choix récupère le fichier dans le répertoire approprié. L'extension *.php* indique que le fichier contient du code PHP. L'interprète PHP intégré au serveur Web est désormais activé. Il analyse le document et lit la balise PHP ouvrante <?php, qui signale le début du code PHP. L'interprète est capable d'exécuter le code PHP et de générer une sortie HTML, fournie par le serveur Web au navigateur.

Si PHP est correctement installé, la page Web suivante devrait apparaître comme **résultat de l'exécution du script** :



Lorsqu'un script est exécuté avec la fonction *phpinfo()*, le navigateur affiche les informations relatives à la configuration de PHP.

La fonction *phpinfo()* est l'abréviation de la valeur par défaut *phpinfo(INFO_ALL)*, qui affiche des informations détaillées sur la configuration PHP de votre serveur Web. Si aucune version de PHP ne peut être trouvée, le navigateur Web affiche un message d'erreur ou transmet le code PHP au navigateur sans l'interpréter.

« Hello World! » ou comment publier du texte grâce à la fonction echo

Si PHP a été correctement installé, vous pouvez écrire votre premier script personnalisé en utilisant l'instruction PHP echo. Contrairement à *phpinfo()*, *echo* n'est pas une fonction au sens propre. Il s'agit plutôt d'une **structure de langage** qui permet de générer une chaîne de caractères subséquente sous forme de texte.

Definition

Les structures de langage sont des instructions utilisées en PHP, qui servent à contrôler le déroulement du programme. Les structures de langage incluent des instructions *echo*, comme *if, for, do, include, return, exit* ou *die*. C'est pourquoi, contrairement aux fonctions, il n'est pas nécessaire d'utiliser des crochets lorsque l'on fait appel à des mots-clés.

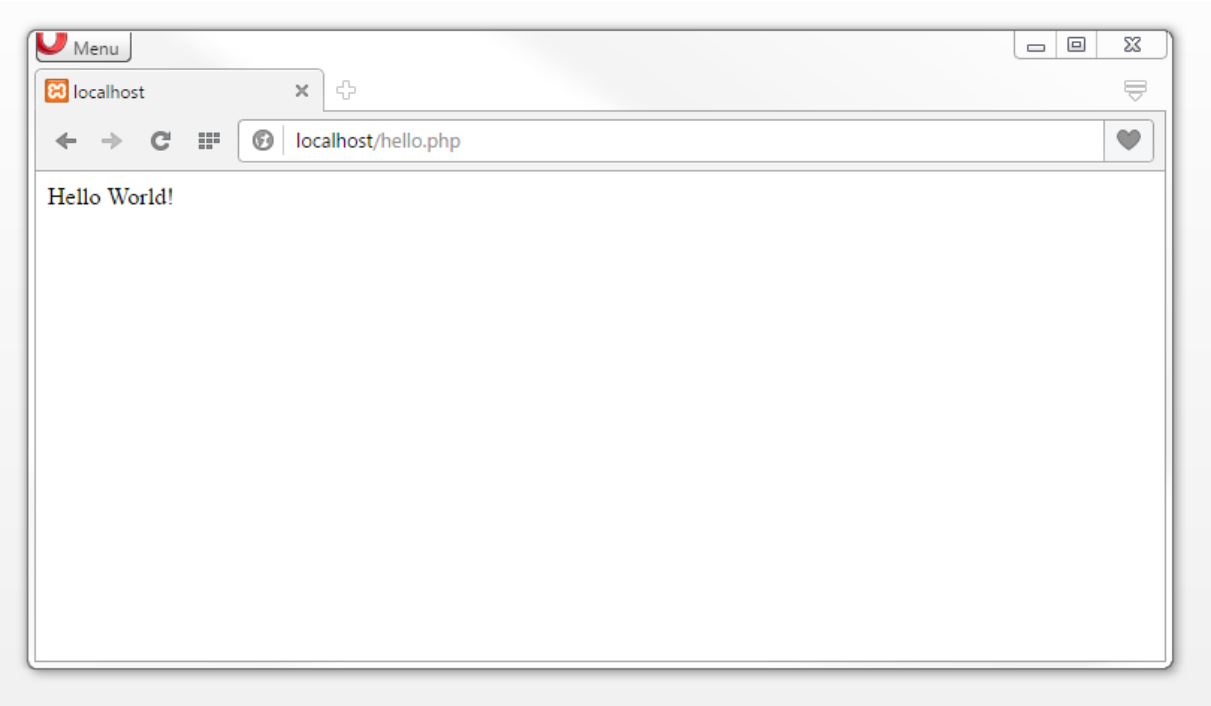
Pour commencer votre nouveau script, ouvrez un nouveau document PHP et inscrivez la ligne de code suivante :

```
<?php

echo 'Hello World!';

?>
```

php

La balise ouvrante <?php débute l'environnement de script. Elle est suivie par la structure de langage *echo*, par la chaîne de caractères *Hello World!* entre **guillemets** et par la balise ?> qui ferme le script. Soyez vigilant au point-virgule après l'instruction. Il est évidemment possible d'utiliser n'importe quel texte à la place de *Hello World!*.

Enregistrez ensuite le script sous le nom *hello.php* dans le dossier *htdocs* sur votre serveur Web, puis ouvrez le fichier via l'URL http://localhost/hello.php dans le navigateur. Si le code a été correctement transféré, la fenêtre du navigateur doit maintenant afficher la chaîne de caractères que vous utilisez :



La structure de langage echo ordonne au serveur Web de publier la chaîne de caractères *Hello World!*.

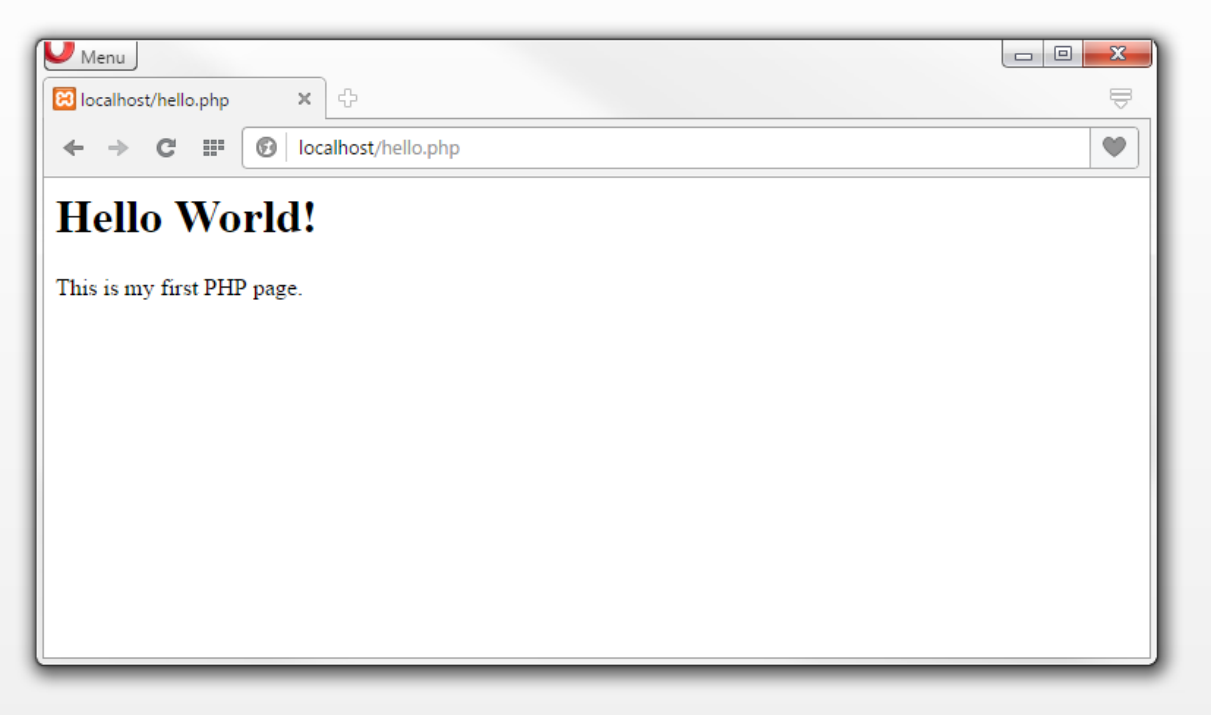
Chaque texte publié grâce à *echo* peut être paramétré avec des **balises HTML** si nécessaire. Ces balises sont ensuite interprétées par le navigateur Web selon la spécification HTML. Jugez par vous-même avec l'exemple de script suivant :

```
<?php
```

```
echo '<h1>Hello World!</h1>

<p>This is my first PHP page.</p>';

?>
```

php

Le **résultat de l'exécution du script** s'affiche ensuite dans le navigateur Web de la façon suivante :



Si le résultat de l'exécution du script contient des balises HTML, elles sont automatiquement interprétées par le navigateur Web.

Grâce à la balise <h1>, la chaîne *Hello World!* est interprétée par le navigateur Web comme un titre de premier niveau. Elle est suivie d'un saut de ligne automatique et du paragraphe de texte <p>.

En fonction des besoins, *echo* peut être utilisée avec des guillemets simples (') et doubles ("). Si vous souhaitez seulement publier du texte, le choix des guillemets n'est pas important. Il le devient en revanche lorsque les variables PHP entrent en jeu.

Les variables

La structure de langage *echo* va bien au-delà de la simple publication d'un texte qui, finalement, pourrait être mise en œuvre sans PHP, sur la base du HTML. La réelle valeur ajoutée d'*echo* repose sur les *variables, ces éléments permettant à l'instruction de créer des textes de manière dynamique*.

Les utilisateurs de PHP rencontrent des variables, par exemple sous la forme suivante : « $exemple ».

Chaque variable se compose d'un **signe dollar ($) suivi du nom de la variable**. On utilise les variables dans les scripts PHP pour intégrer des données externes dans des pages Web. Il peut s'agir de valeurs de différents types, de simples nombres et chaînes de caractères à des textes entiers, en passant par des structures de document HTML.

Le langage PHP compte sept types de variables :

Type de variable	Description
String	String, qui signifie corde, correspond à une chaîne de caractères. Il peut s'agir d'un mot, d'une phrase, texte ou du code HTML complet d'une page Web.
Integer	Un integer est un entier sans décimales. Il peut être positif ou négatif.
Float	Un float est un nombre à virgule flottante, c'est-à-dire une valeur numérique avec décimales. En langa programmation, la virgule est représentée sous la forme d'un point (.). PHP prend en charge jusqu'à 14 décimales.
Boolean	Les variables boolean sont le résultat d'une opération logique et ne peuvent avoir que deux valeurs : *TRUE* (vrai) et *FALSE* (faux). On a recours à ce type de variable par exemple lorsque l'on utilise des conditions.
Array	Un array est une variable pouvant contenir plusieurs éléments : il s'agit d'un regroupement de plusieu données de structure similaire, rassemblées pour former un tableau.
Object	La variable *object* permet aux programmeurs de définir leurs propres types de données. Elle est utilisé la programmation orientée objet. La variable *object* n'est pas abordée dans ce tutoriel PHP pour début
NULL	NULL indique une variable ne représentant aucune valeur.

En général, le contenu est géré de façon centralisée, à l'aide de **systèmes de bases de données**. Il est également possible de définir les valeurs des variables directement dans le script. Ce type d'affectation fonctionne selon le schéma suivant :

$exemple = "Valeur";

php

Le symbole dollar caractéristique est suivi du nom de la variable (dans ce cas, *exemple*) et est connecté par le **signe égal (=)** à une valeur encadrée de doubles guillemets. Les valeurs des variables de type Integer et Float sont inscrites sans guillemets, de la façon suivante : *$exemple = 24;* ou *$exemple = 2.7;*.

PHP offre une grande liberté en ce qui concerne le nom des variables, que vous pouvez nommer à votre guise tant que les contraintes suivantes sont respectées :

- Chaque variable doit commencer par le symbole dollar.
- Un nom de variable est une chaîne composée de lettres, de chiffres et de tirets, du type : *$exemple_1*.
- Un nom de variable valide commence toujours par une lettre ou un tiret, comme suit : *$exemple1* ou *$_exemple*, mais jamais par un chiffre (faux : *$1exemple*).
- PHP est sensible à la casse : le langage de script fait une distinction entre majuscules et minuscules. *$exemple* est donc différent de *$Exemple*.
- Le nom de la variable ne doit contenir ni espaces ni sauts de ligne (faux : *$exemple 1*).

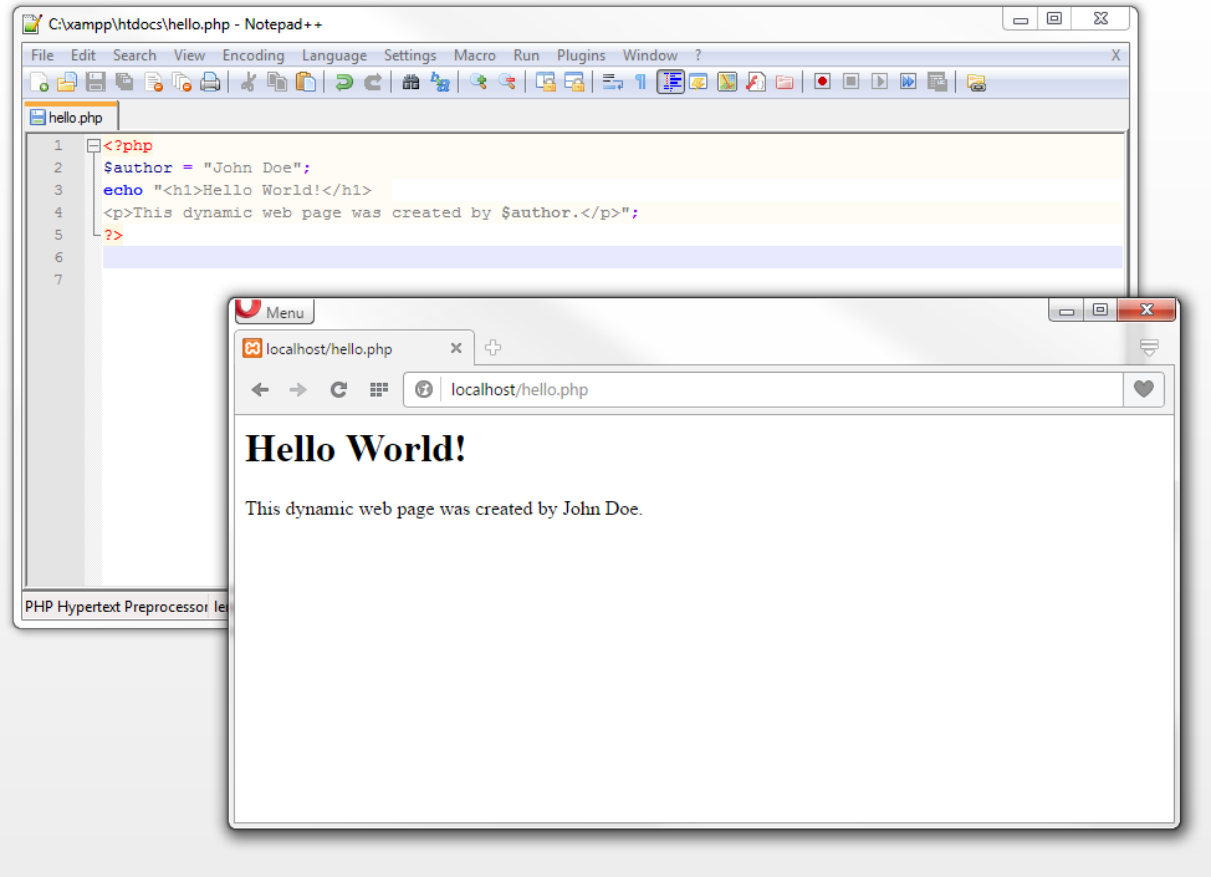
- Les chaînes de caractères que PHP réserve à d'autres fins ne peuvent pas être personnalisées (par exemple *$this*).

Voici un exemple :

```
<?php

$author = "John Doe";

echo "<h1>Hello World!</h1>

<p>This dynamic web page was created by $author.</p>";

?>
```

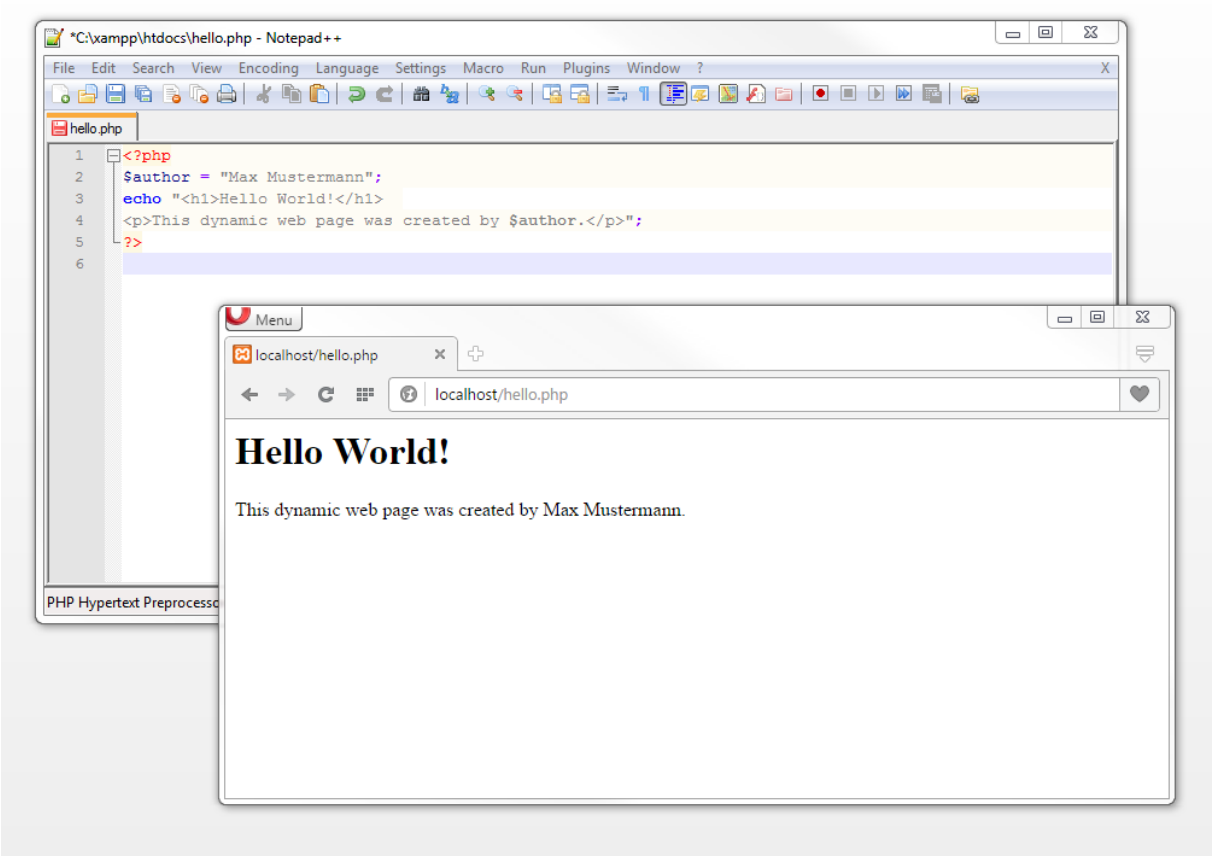
php

La balise PHP ouvrante est suivie de la **définition de la variable** : on utilise la valeur *John Doe* pour *$author*. Pendant l'exécution du script, la variable *$author* est remplacée par la valeur *John Doe* à chaque fois qu'elle est spécifiée dans l'environnement du script. L'image ci-dessous illustre la manière dont ceci est présenté dans le navigateur.



Lors de l'exécution du script, la variable $author a été remplacée par la valeur John Doe.

Si la page Web n'a pas été créée par John Doe mais par son collègue allemand Max Mustermann, modifiez simplement la variable *$author* pour corriger l'erreur.



La variable $author a été remplacée par la valeur Max Mustermann.

Ceci est particulièrement efficace si une variable intervient plus d'une fois dans un script. Dans ce cas, une seule modification suffit pour en corriger toutes les occurrences, puisque c'est ici que la valeur de la variable est définie.

C'est là que réside la puissance de PHP : le contenu peut être intégré sous forme de variables. Cette fonctionnalité constitue la base du développement Web dynamique. À l'inverse des pages Web statiques, disponibles sous forme de pages HTML prêtes à l'emploi, les **pages Web dynamiques** ne sont créées que lorsque l'utilisateur cherche à les afficher. L'interprète PHP utilise les variables pour charger les éléments individuels de la page Web à laquelle on souhaite accéder : pour ce faire, il a recours à différentes bases de données dont il assemble les éléments dans une page HTML adaptée à une requête spécifique.

Les avantages de ce concept sont évidents : lorsque des éléments du site Web (par exemple dans la zone de pied de page) sont modifiés, il n'est pas nécessaire de procéder à des ajustements manuels sur chaque sous-page du projet Web. Il suffit au contraire de **mettre à jour l'entrée en question dans la base de données**. De cette façon, la révision est automatiquement adoptée pour toutes les pages Web qui incluent les données correspondantes en tant que variables. Si une **variable est définie plusieurs fois dans un script**, chaque nouvelle définition remplace la précédente.
L'instruction *echo* met toujours en œuvre la valeur actuelle d'une variable.

```
<?php

$author = "John Doe";

echo "<h1>Hello World!</h1>
```

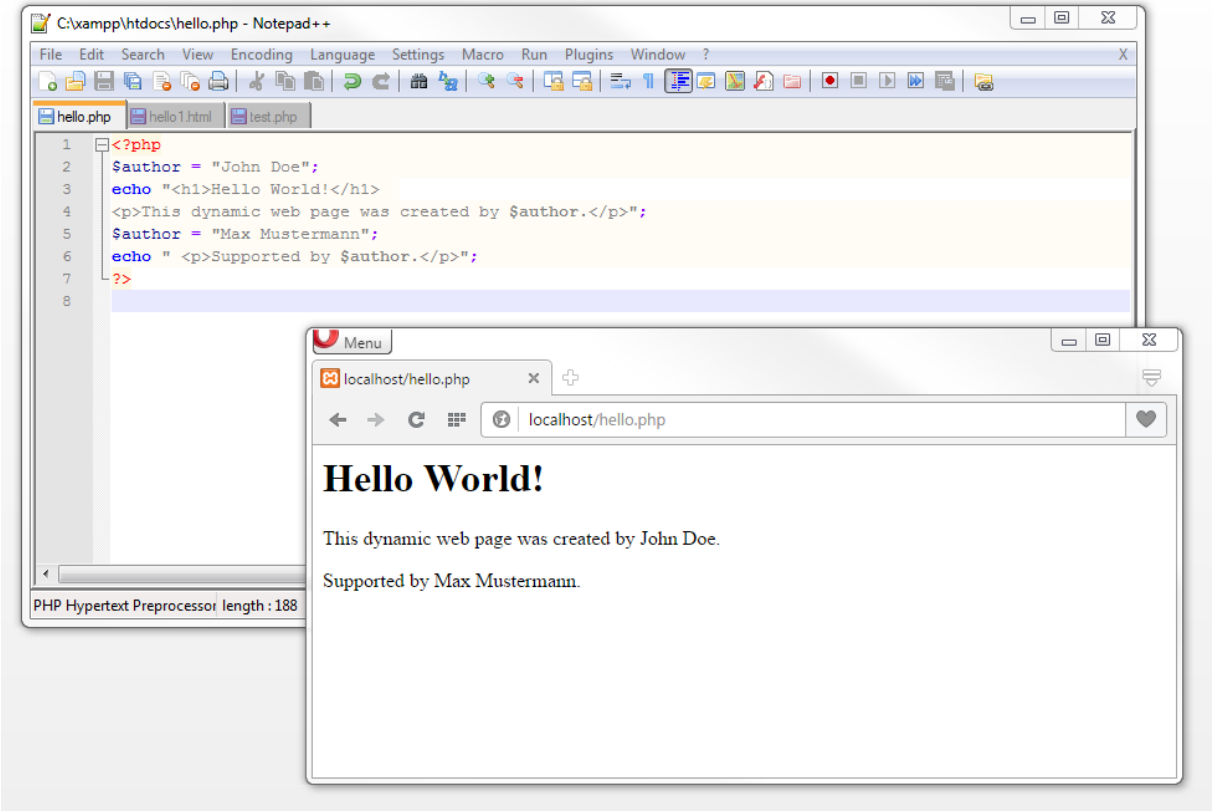
This dynamic web page was created by $author. ";

$author = "Max Mustermann";

echo " Supported by $author. ";

?>

php



La valeur John Doe est remplacée par la valeur Max Mustermann.

Dans l'exemple de code, la variable *$author* reçoit d'abord la valeur *John Doe*, qui est ensuite remplacée par *Max Mustermann*.

En ce qui concerne les **guillemets**, les règles sont différentes. En effet, à l'inverse des chaînes de caractères, les variables individuelles n'ont pas besoin d'être citées entre guillemets :

<?php

$author = "John Doe";

echo $author;

?>

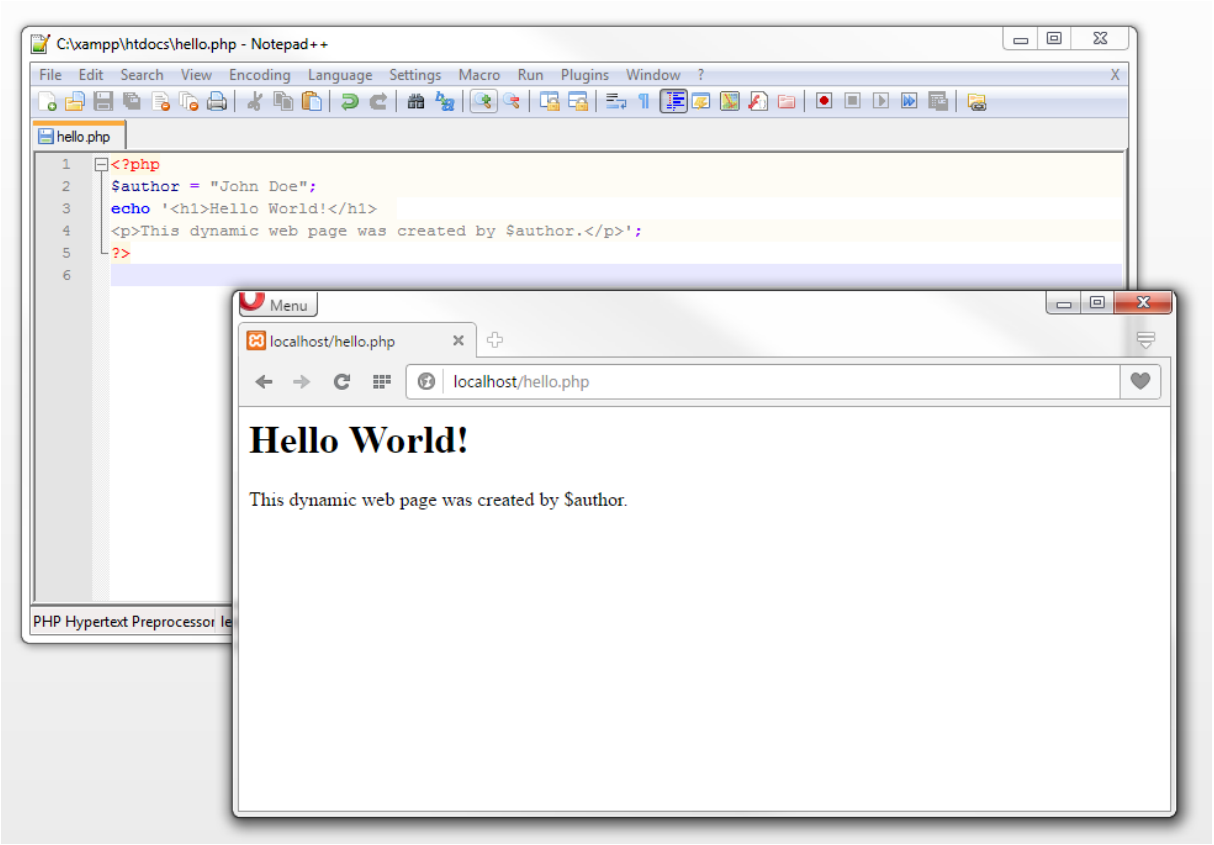
php

Ceci n'est pas valable si la variable est utilisée dans une chaîne de caractères. Dans ce cas, utilisez les doubles guillemets ("). Ceux-ci permettent d'indiquer à l'interprète PHP qu'il doit chercher dans la chaîne de caractères des variables pouvant être remplacées par les valeurs associées. Les chaînes de

caractères entre guillemets simples (’) sont interprétées et affichées comme des informations en texte brut, bien qu’il s’agisse de variables.

```
<?php

$author = "John Doe";

echo '<h1>Hello World!</h1>

<p>This dynamic web page was created by $author.</p>';

?>
```

php



Les guillemets simples permettent de publier du texte brut. Les variables ne sont pas interprétées.

Si vous oubliez complètement les guillemets, PHP signale une erreur de syntaxe.

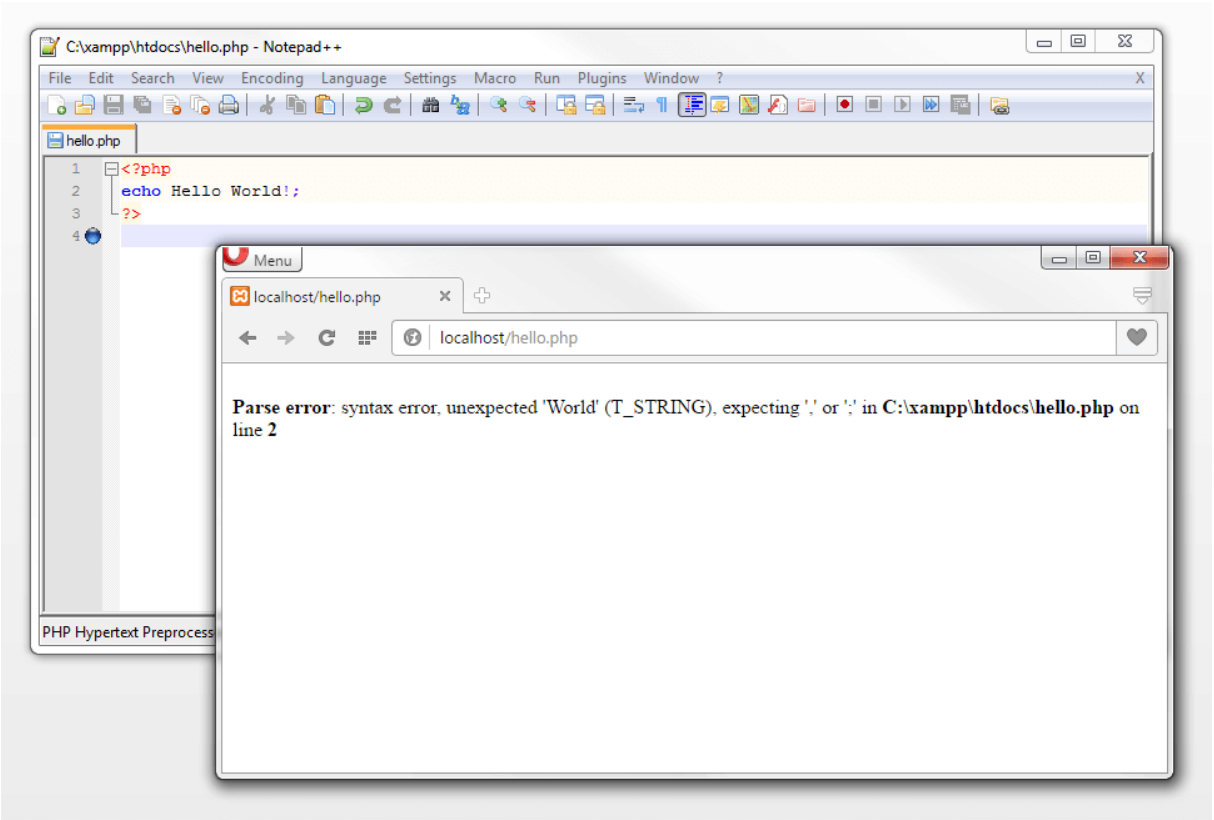
Messages d’erreur et masquage

En cas d’**erreurs syntaxiques**, le code PHP n’est pas valide et l’interprète PHP émet un message d’erreur. Par exemple, ceci se produit si l’on utilise l’instruction *echo* avec une chaîne de caractères sans guillemets :

```
<?php

echo Hello World!;

?>
```

php

En général, les messages d'erreur comportent des informations sur l'origine d'une erreur et fournissent des éléments significatifs pour la corriger.



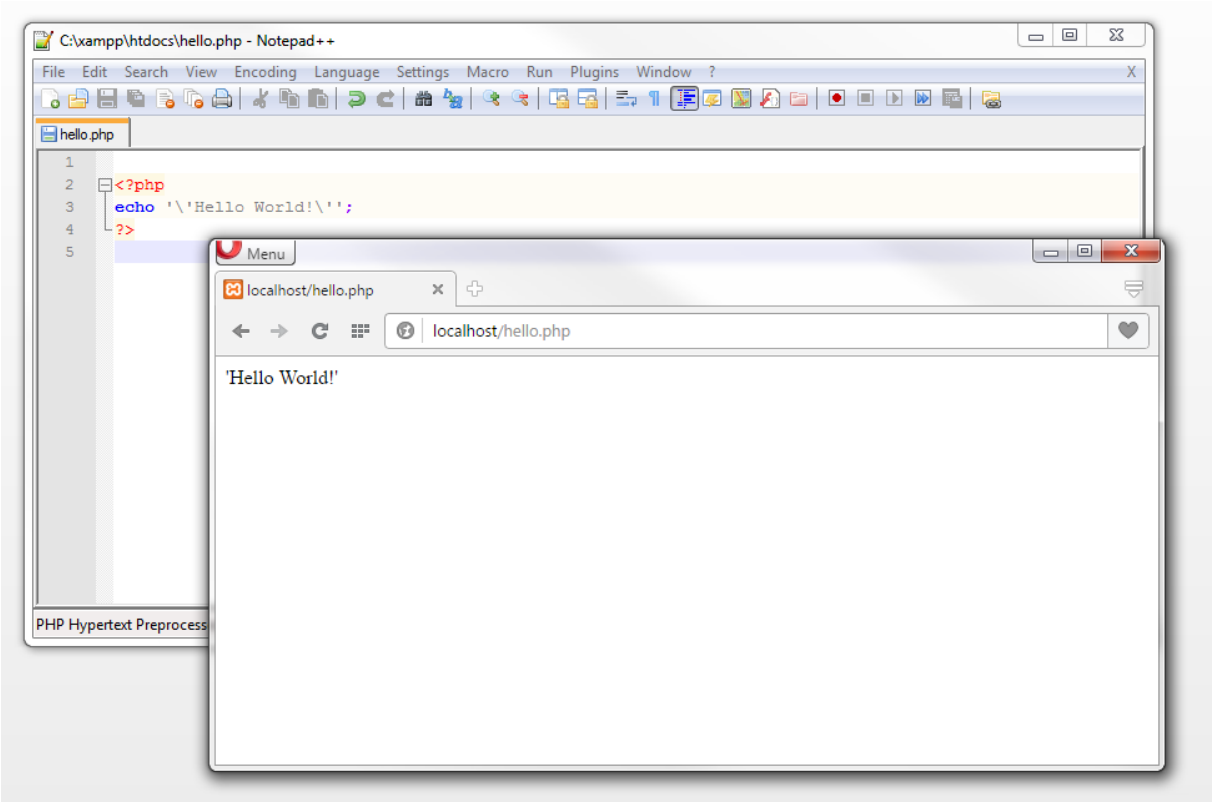
Un message d'erreur indique une erreur de syntaxe dans le code du programme.

L'exemple indique une erreur à la ligne 2 de notre code du programme, soit exactement là où nous avons volontairement oublié les guillemets.

Des **erreurs de syntaxe** se produisent également lorsque vous publiez des caractères sous forme de texte qui sont associés à une tâche particulière en PHP, les guillemets notamment ('). En PHP, les caractères de ce type ne peuvent être publiés de la même manière qu'un texte seulement si vous informez l'interprète que le caractère en question n'est plus contraint d'assumer sa tâche initiale. Dans le cas de guillemets simples, il existe deux options : vous pouvez placer une chaîne de caractères avec des guillemets simples entre guillemets doubles ou masquer les guillemets avec un backslash (\), comme suit :

```
<?php

echo '\'Hello World!\' ';

?>
```

php



Lorsque des caractères sont masqués par un backslash, ils sont dispensés de leurs fonctions initiales.

Il est même possible de combiner des guillemets simples et doubles :

<?php

echo " 'Hello World!' ";

?>

php

Mais pas de la façon suivante :

<?php

echo ' 'Hello World!' ';

?>

php

Pour plus de lisibilité des exemples, des espaces ont été insérés entre les guillemets, mais il ne faut pas en ajouter dans la pratique.

Les opérateurs de concaténation

Si vous souhaitez publier simultanément plusieurs variables dans un script PHP, vous pouvez procéder de la façon suivante, expliquée précédemment :

<?php

$author1 = "John Doe";

$author2 = "Max Mustermann";

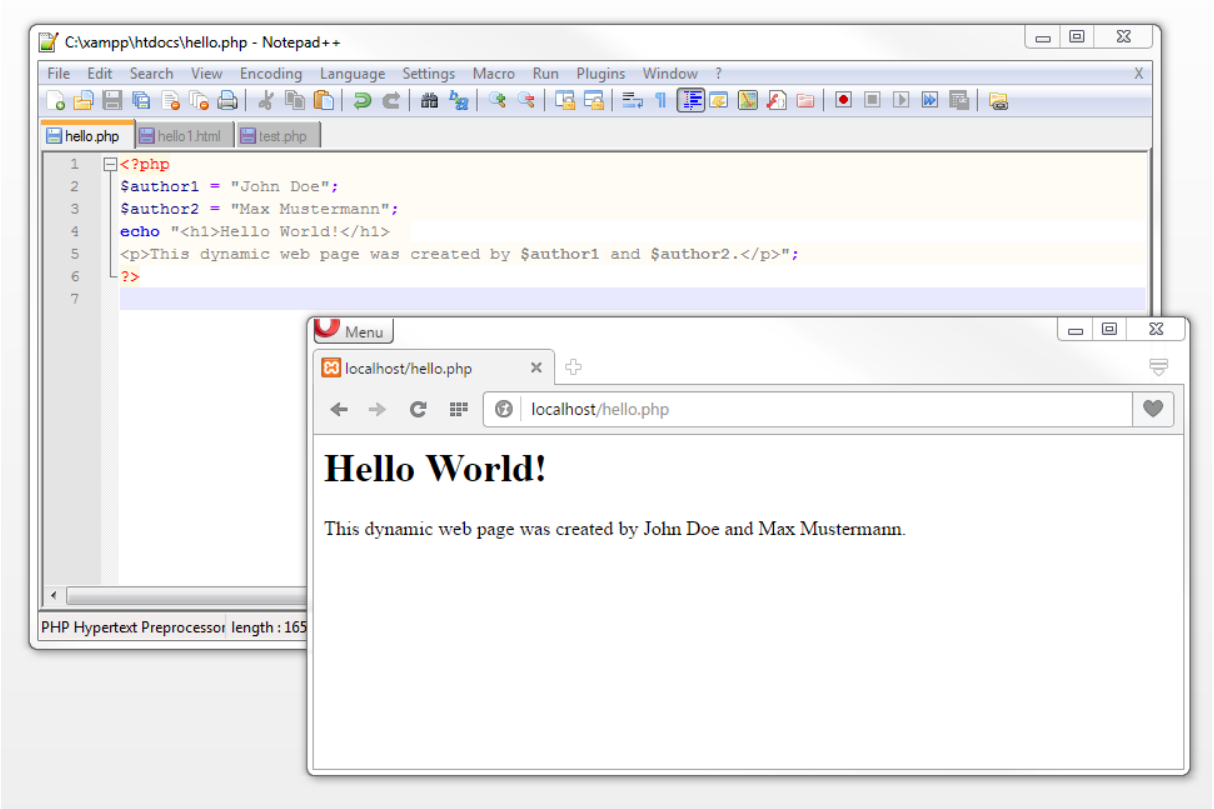
echo "<h1>Hello World!</h1>

This dynamic web page was created by $author1 and $author2. ";

?>

php

Il vous suffit d'écrire les deux variables avec le reste du texte à publier dans la chaîne de caractères placée entre guillemets doubles. PHP détecte automatiquement les variables grâce au signe dollar ($) et insère les valeurs correspondantes.



PHP n'accepte les variables de la chaîne que si le script ne contient pas de fonctions.

Cependant, cette méthode est mal vue par la plupart des programmeurs, car le principe le plus répandu veut que **les variables ne fassent pas partie de la chaîne de caractères**. Ceci s'explique notamment par le fait que cette séparation est prescrite dans de nombreux langages de programmation. En outre, PHP exige également la séparation de la chaîne et de la variable lorsque l'on travaille avec des appels de fonction ou des variables plus complexes. Il est donc conseillé d'appliquer systématiquement cette séparation, même dans le cas d'une publication de texte brut, bien que ce ne soit pas absolument nécessaire.

Lorsque l'on travaille avec des variables, il est essentiel de gérer plusieurs éléments liés entre eux pour la production. En PHP, c'est le rôle de l'**opérateur de concaténation** (.).

Le code de l'exemple ci-dessus doit ressembler à ceci s'il est programmé de façon « propre » :

<?php

$author1 = "John Doe";

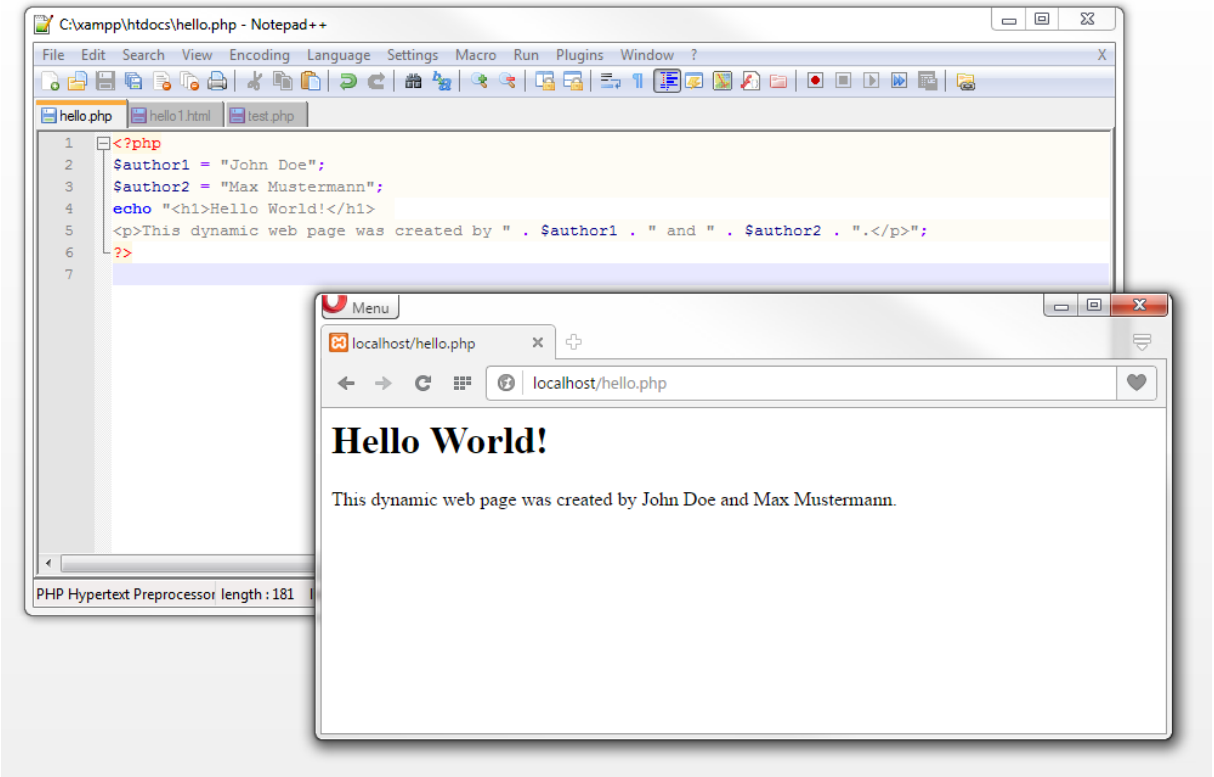
$author2 = "Max Mustermann";

echo '<h1>Hello World!</h1>

This dynamic web page was created by ' . $author1 . ' and ' . $author2 . '. ';

?>

php



Les opérateurs de concaténation relient les chaines de caractères et les variables.

Il s'agit donc de trois chaînes de caractères et deux variables qui sont concaténés en une seule chaîne.

Chaîne 1		Variable 1		Chaîne 2		Variable 2		Cha
'<h1>Hello World!</h1> <p>This dynamic web page was created by '	.	*$author1*	.	*' and '*	.	*$author2*	.	*'.<*

Il est à noter qu'un opérateur de concaténation assemble les chaînes de caractères ou les variables sans espaces. Si une espace est nécessaire, elle doit être intégrée entres les guillemets, comme dans l'exemple.

Les programmeurs utilisent l'opérateur de concaténation non seulement pour connecter les chaînes et les variables lors de l'affichage du texte, mais également pour apporter des extensions aux variables. L'exemple suivant illustre cette utilisation :

<?php

$example = 'Hello ';

$example .= 'World';

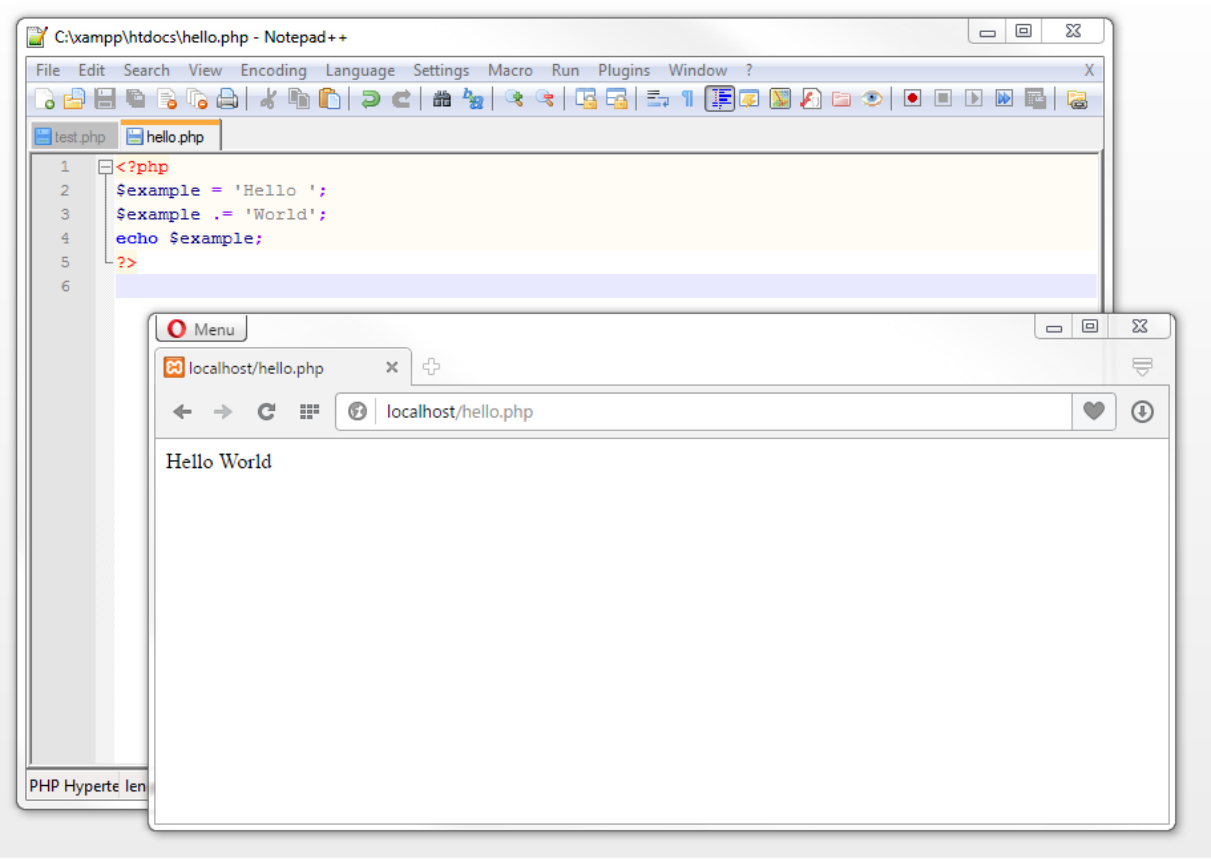
echo $example;

?>

php

Pour étendre la valeur d'une variable, redéfinissez-la, mais ajoutez également l'opérateur de concaténation *point (.)* avant le signe égal. C'est l'**abréviation courante** pour *$example = $example . 'World'*.

PHP ajoute la nouvelle valeur à celle définie au préalable. Si vous souhaitez insérer une espace entre les deux valeurs, insérez-la à la fin de la première chaîne de caractères, comme dans l'exemple.



La chaîne Hello a été étendue en Hello World.

Insérer du PHP en HTML

L'interprète PHP déchiffre uniquement le code compris entre les balises PHP ouvrante et fermante :

<?php [Cette chaîne est interprétée par PHP] ?>

Toutes les autres sections du document sont ignorées par l'interprète et transmises au serveur Web telles quelles. Le code PHP peut ainsi être **intégré à des documents HTML**, au choix, par exemple pour créer un modèle de système de gestion de contenu. Assurez-vous que les documents HTML contenant du code PHP sont enregistrés dans des fichiers PHP. Sinon, le document n'est pas prétraité par l'interprète PHP, mais livré directement au navigateur Web, ce qui ferait apparaître le code du programme sous forme de texte sur le site Web. On peut donc imaginer l'interprète PHP comme un collègue un peu paresseux du serveur Web, qui ne travaille que lorsqu'on lui demande de le faire, par

exemple en ouvrant une balise PHP. Si vous souhaitez **combiner HTML et PHP**, rédigez la page HTML selon la structure classique du document, puis enregistrez-la sous l'extension de fichier *.php* :

```
<!DOCTYPE html>

<html lang="de">

  <head>

    <meta charset="utf-8">

    <title>My first PHP page</title>

  </head>

    <body>

  <h1>Hello World</h1>

  <p>What is the current time and date?</p>

  </body>

</html>
```

html

Ajoutez ensuite un script PHP dans votre document HTML. Il s'agit de veiller à ce que l'intégralité du code du programme se trouve à l'intérieur des balises PHP.

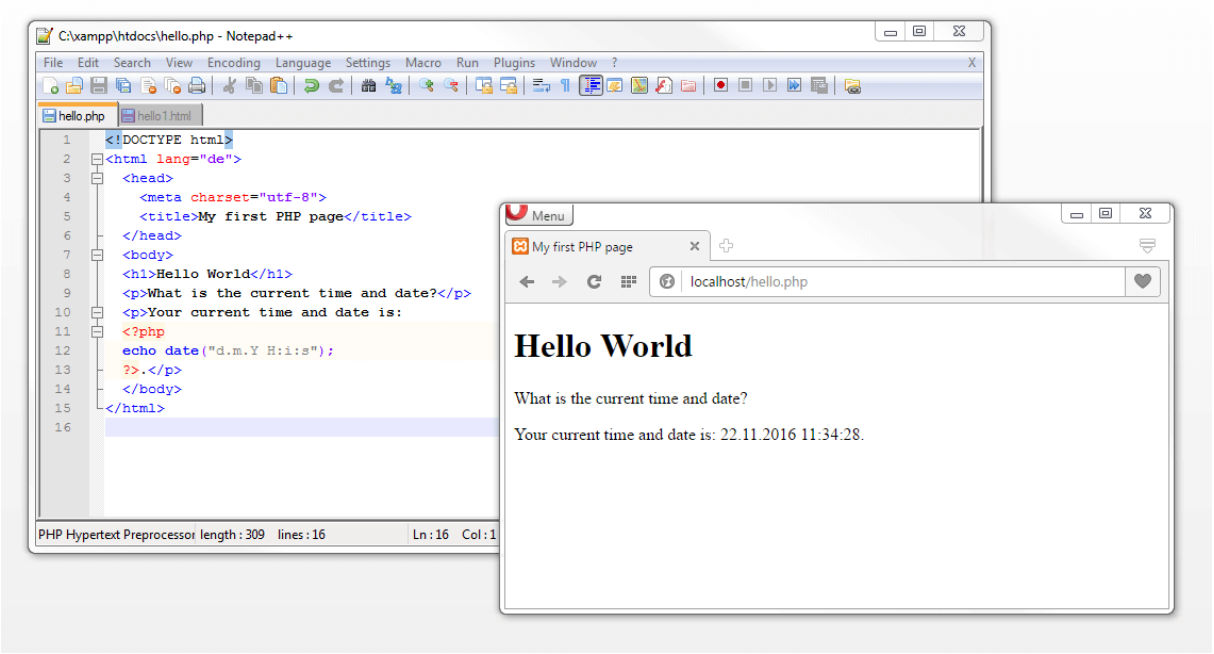
```
<!DOCTYPE html>

<html lang="de">

  <head>

    <meta charset="utf-8">

    <title>My first PHP page</title>

  </head>

  <body>

  <h1>Hello World</h1>

  <p>What is the current time and date?</p>

  <p>Your current time and date is:

  <?php

  echo date("d.m.Y H:i:s");

  ?>.</p>

  </body>

</html>
```

html

Dans l'exemple, nous avons combiné la structure de langage *echo* avec la fonction PHP date() pour afficher la date et l'heure actuelles sous forme de texte côté serveur. Le paramètre de la fonction spécifie le format souhaité sous la forme d'une chaîne de caractères :

d.m.Y H:i:s = jour.mois.année heure:minute:seconde

Si un navigateur Web sollicite ce type de fichier, l'interprète PHP exécute d'abord le script, puis écrit l'**heure actuelle sous forme de texte** dans le document HTML. L'heure est ensuite transmise grâce au serveur Web et affichée dans le navigateur en tant que page Web.



Le document HTML contient un script PHP intégré, qui donne la date et l'heure actualisées.

La fonction commentaires de PHP

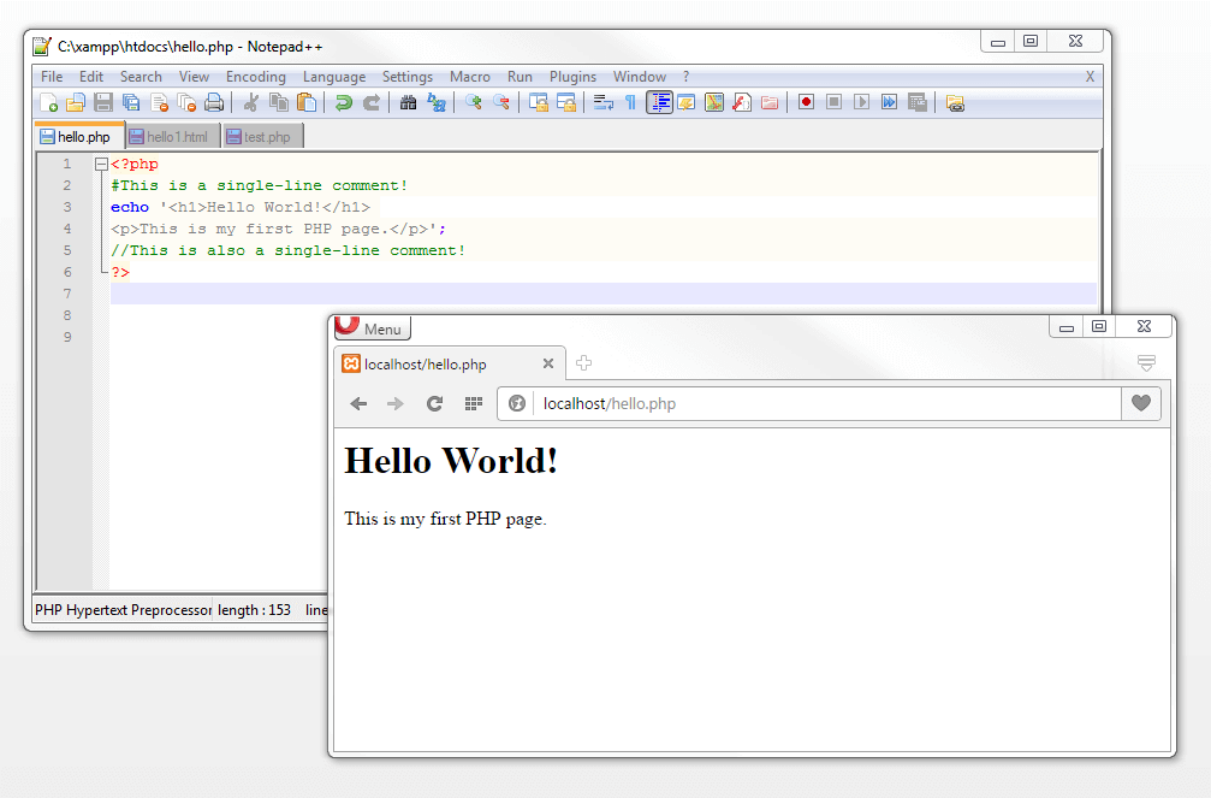
De même que le code HTML, PHP peut être commenté à volonté. Tant que cela est signalé dans la syntaxe, les **commentaires dans le code source** sont ignorés par l'interprète PHP. PHP propose trois alternatives différentes.

Si vous souhaitez identifier une ligne entière comme un commentaire, et donc l'exclure de l'interprétation, vous devez utiliser le **hashtag (#)** ou deux **barres obliques à la suite (//)**. Dans l'exemple de code suivant, ces deux options sont appliquées :

```
<?php

#This is a single-line comment!

echo '<h1>Hello World!</h1>

<p>This is my first PHP page.</p>';

//This is also a single-line comment!

?>
```

php

L'éditeur de texte Notepad++ affiche les commentaires en vert. Les passages de texte identifiés comme des commentaires dans l'environnement de script n'arrivent pas dans le navigateur Web, à l'inverse des commentaires HTML, car ils sont ignorés par l'interprète PHP lors de l'exécution du script.



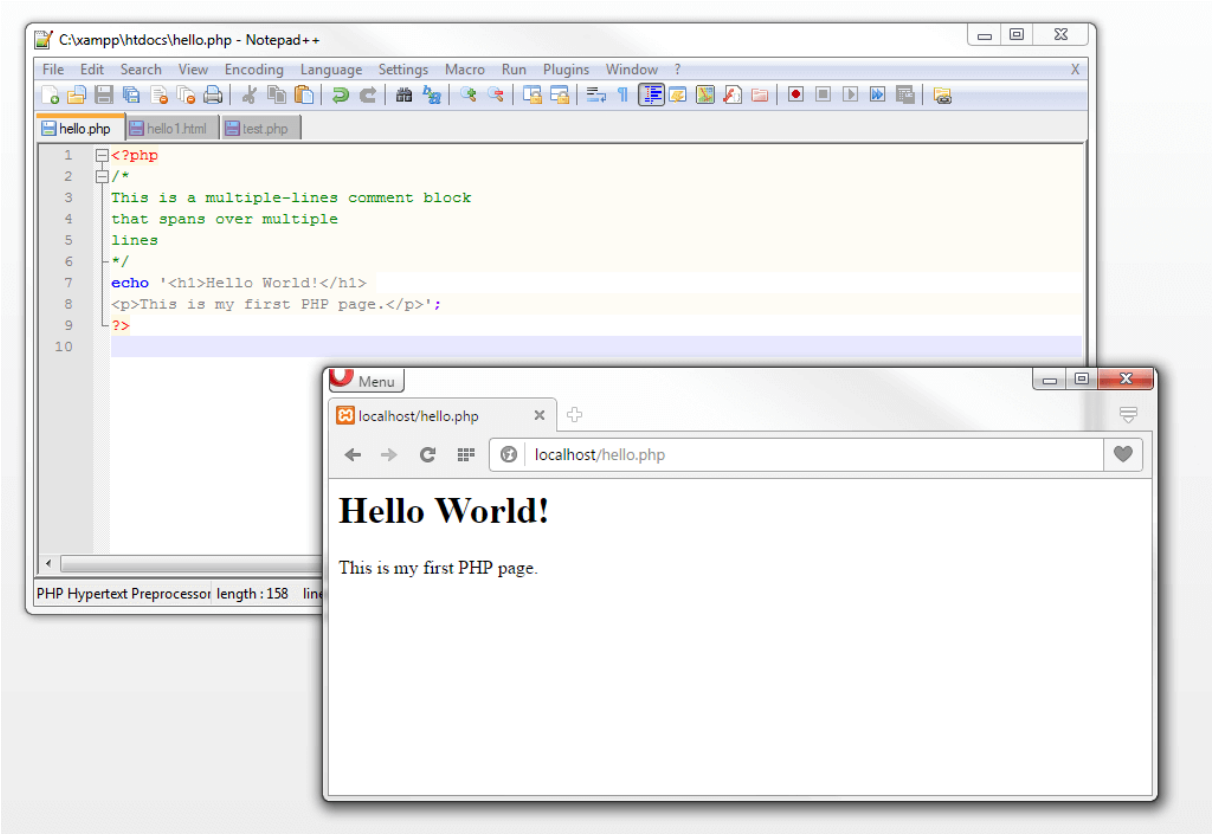
Les lignes indiquées en tant que commentaires ne sont pas visibles dans le document final.

Vous pouvez également insérer des commentaires **sur plusieurs lignes**. Pour ce faire, marquez le début d'une section de commentaire par une barre oblique suivie d'un astérisque (/*) et la fin par un astérisque suivi d'une barre oblique (*/).

```
<?php
/*
This is a multiple-lines comment block
that spans over multiple
lines
*/
echo '<h1>Hello World!</h1>
<p>This is my first PHP page.</p>';
?>
```

php

Ces commentaires ainsi marqués ne sont pas analysés et n'apparaissent donc pas sur le site Web.



Le commentaire de plusieurs lignes n'est pas visible dans le résultat final.

Les programmeurs utilisent des commentaires pour structurer le code source de leurs scripts, pour laisser des notes pour une modification ultérieure ou pour ajouter des informations internes, comme l'auteur ou la date.

Les commentaires sont optionnels et doivent être **utilisés avec parcimonie** pour assurer une bonne lisibilité du code source.

Calculer avec des variables

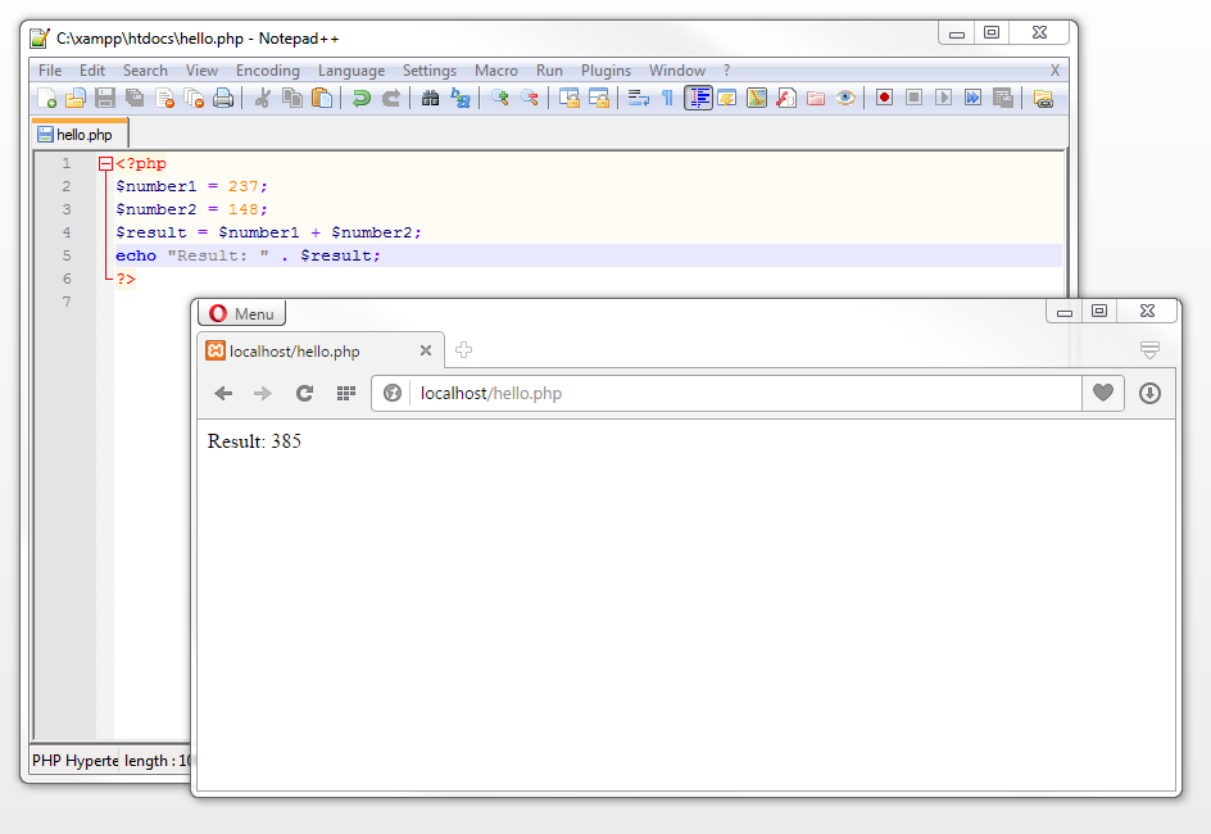
Dans le cadre de ce tutoriel PHP, nous avons déjà présenté les variables. Nous leur avons jusqu'à présent surtout assigné des valeurs de chaînes de caractères. Nous allons maintenant traiter les variables représentant des **nombres entiers** (integers) ou des **nombres à virgule flottante** (floats).

Si les variables sont dotées de valeurs numériques, PHP offre la possibilité d'effectuer des calculs. Commençons par une simple addition de deux nombres entiers :

```
<?php

$number1 = 237;

$number2 = 148;

$result = $number1 + $numer2;

echo "result: " . $result;

?>
```

php

Tout d'abord, nous attribuons aux variables *$number1 et $number2* les entiers 237 et 148, puis définissons une variable *$result*, qui correspond à la somme des variables *$number1 et $number2*. Pour ce faire, nous utilisons l'**opérateur arithmétique +**(plus). Enfin, nous utilisons la structure de langage*echo* pour afficher le résultat de cette addition sous forme de texte. Notez qu'il n'est pas nécessaire d'utiliser des guillemets lorsque l'on attribue des valeurs numériques aux variables.



Le résultat de l'addition est affiché dans le navigateur comme du texte.

L'exemple de code suivant présente une **sélection de calculs mathématiques** qui peuvent être effectués avec PHP côté serveur. Les opérateurs PHP utilisés correspondent essentiellement aux caractères mathématiques normalisés.

Opérateur arithmétique	Opération	Résultat
$number1 + $number2	Addition	Somme de *$number1 et $number2*
$number1 - $number2	Soustraction	Différence de *$number1 et $number2*
$number1 *$number2	Multiplication	Produit de $number1 et $number2
$number1 / $number2	Division	Quotient de *$number1 et $number2*
$number1 **$number2	Puissance	Puissance de *$number2* par *$number1*

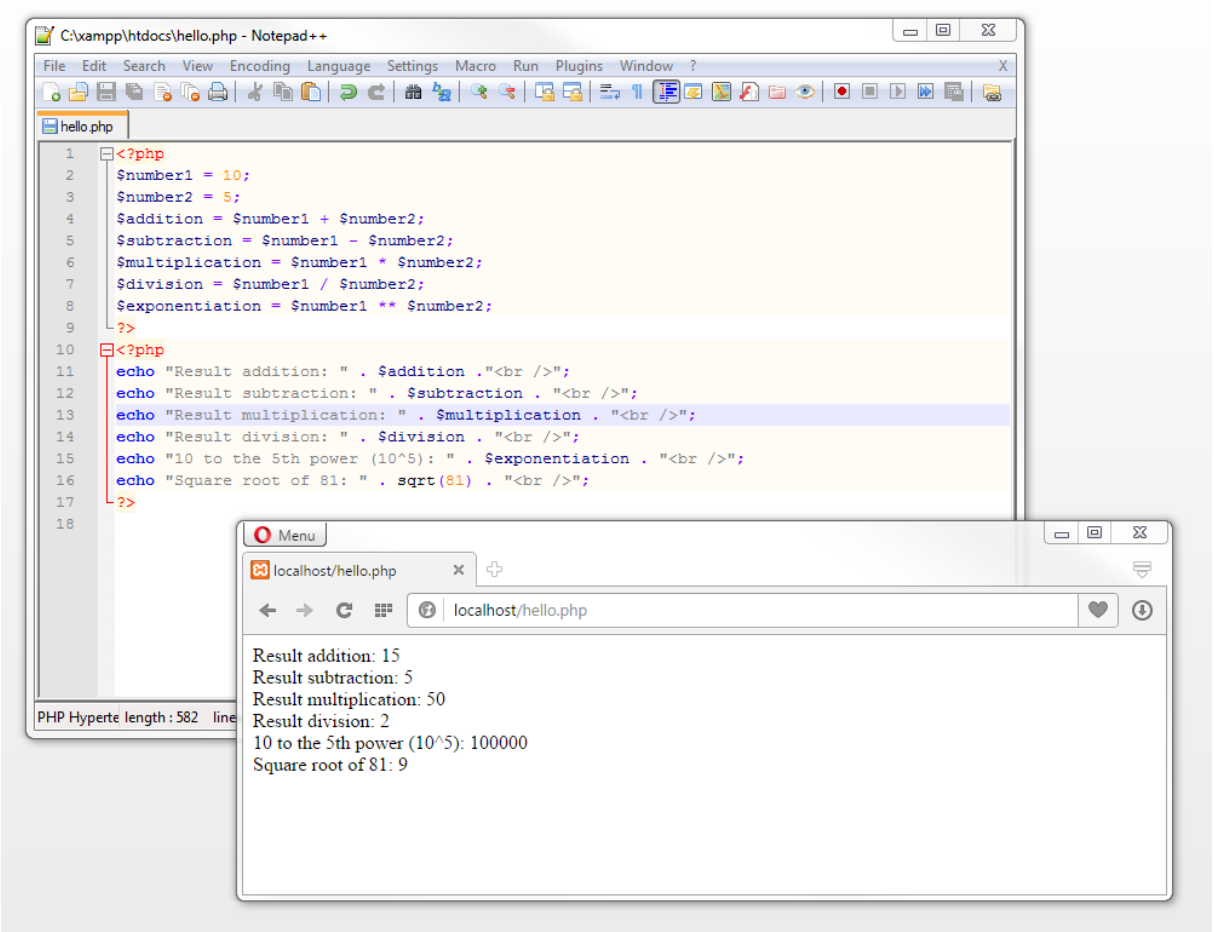
```
<?php

$number1 = 10;

$number2 = 5;
```

```
$addition = $number1 + $number2; //Addition

$subtraction = $number1 - $number2; //Subtraction

$multiplication = $number1 * $number2; //Multiplication

$division = $number1 / $number2; //Division

$exponentiation = $number1 ** $number2; //Exponentiation

?>
```

php

```
<?php

echo "Result of addition: " . $addition ."<br />";

echo "Result of subtraction: " . $subtraction . "<br />";

echo "Result of multiplication: " . $multiplication . "<br />";

echo "Result of division: " . $division . "<br />";

echo "10 to the 5th power (10^5): " . $exponentiation . "<br />";

echo "Square root of 81: " . sqrt(81) . "<br />";

?>
```

php



Aperçu des résultats des calculs

Pour les calculs complexes, il est possible de combiner les différents opérateurs arithmétiques dans un même script :

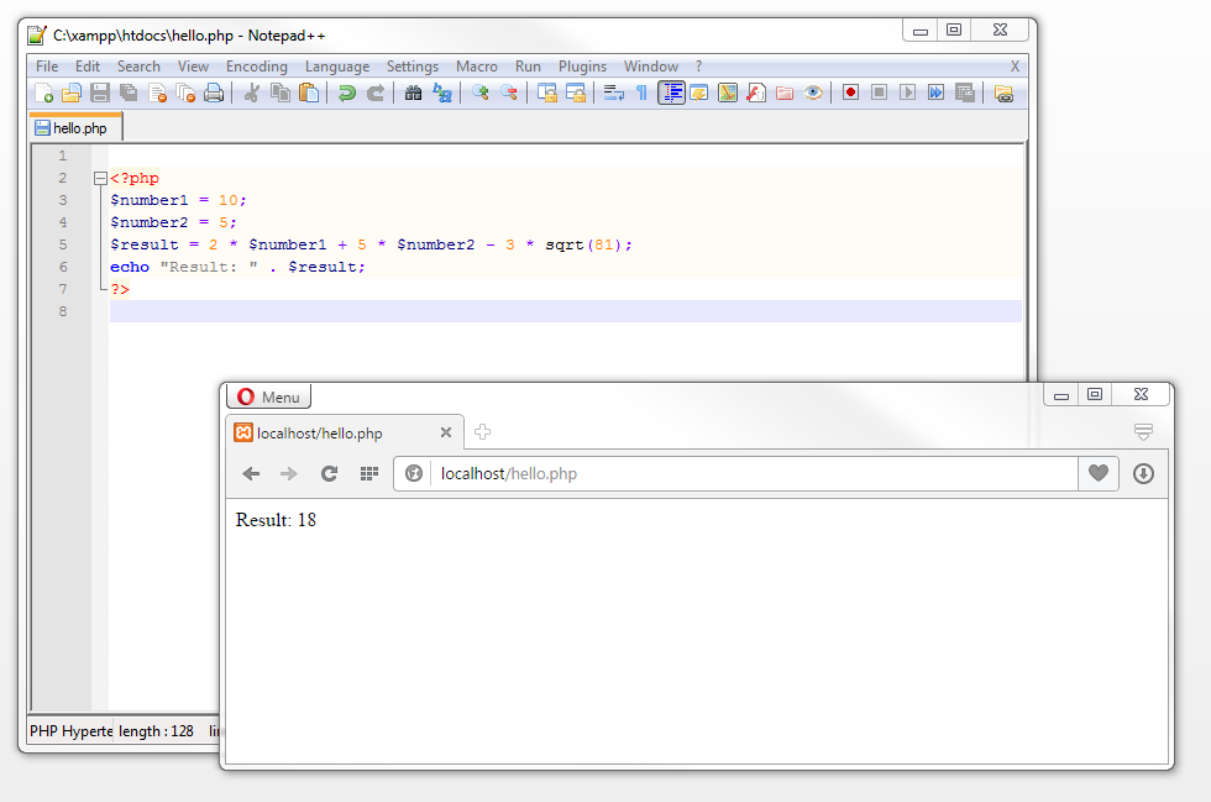
```
<?php

$number1 = 10;

$number2 = 5;

$result = 2 *$number1 + 5* $number2 - 3 * sqrt(81);

echo "Result: " . $result;

?>
```

php

L'interprète PHP détermine les valeurs des variables et les calcule ainsi :

2 *10 + 5* 5 - 3 * √81 = 20 + 25 – 27 = 18

La fonction *sqrt()* calcule la racine carrée du paramètre entre parenthèses. L'**ordre traditionnel des opérateurs** en mathématiques s'applique : les multiplications sont prioritaires sur les additions ou soustractions. L'instruction *echo* renvoie le résultat sous la forme d'une chaîne de caractères pour le navigateur Web.



PHP applique la règle de priorité de la multiplication sur les additions et soustractions.

En PHP, les **termes entre parenthèses** sont aussi analysés en priorité. Voici maintenant un exemple avec des nombres à virgule flottante :

<?php

$number1 = 2.5;

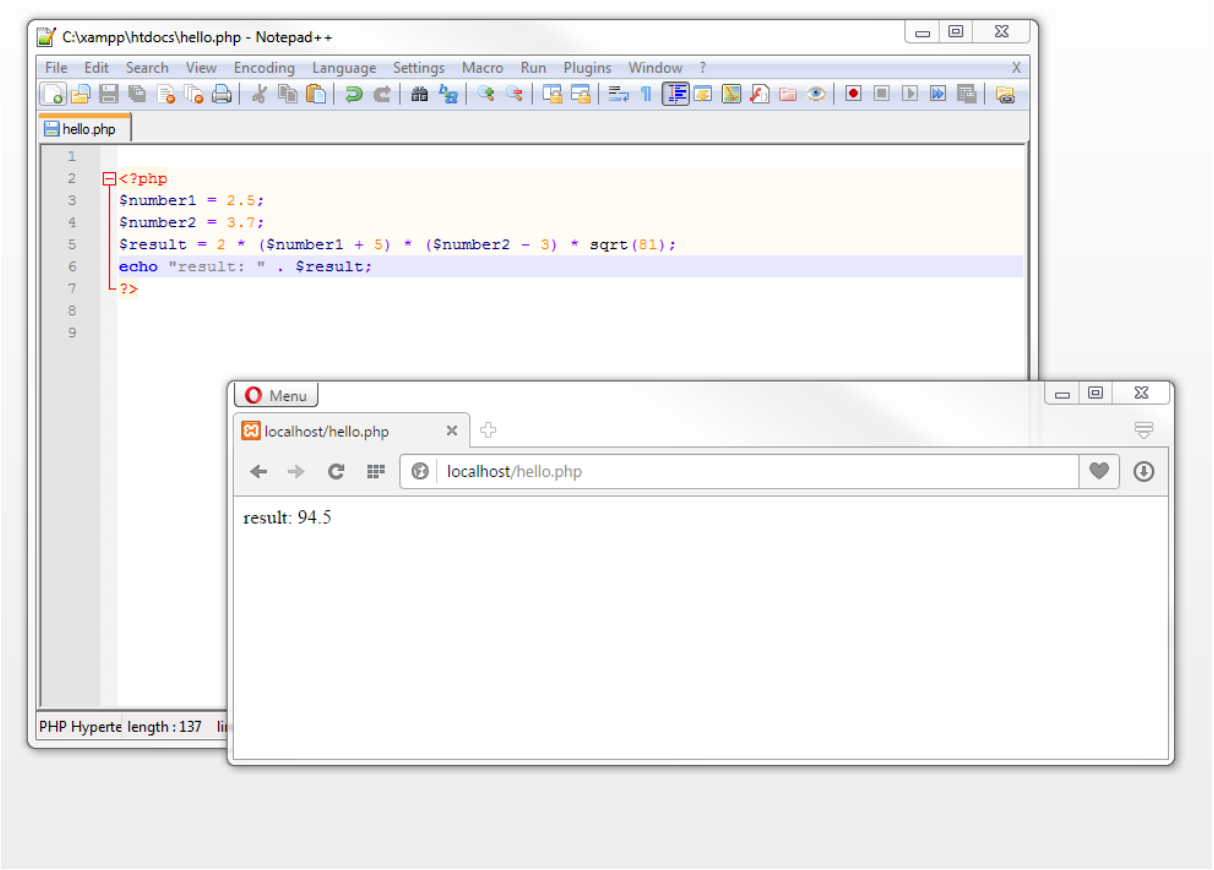
$number2 = 3.7;

$result = 2 *($number1 + 5)* ($number2 - 3) * sqrt(81);

echo "Result: " . $result;

?>

php



À l'instar de tous les langages de programmation courants, PHP permet aux opérateurs d'**augmenter ou de diminuer les valeurs numériques de 1**. On distingue l'opérateur pré-incrémentation, l'opérateur pré-décrémentation, l'opérateur post-incrémentation et l'opérateur post-décrémentation.

Opération	Opérateur	Résultat
Pré-incrémentation	*++$number*	L'opérateur *++* incrémente la valeur de la variable *$number*. La valeur est augmenté Le résultat est restitué sous la forme d'une nouvelle valeur de $number.
Pré-décrémentation	*–$number*	L'opérateur – diminue la valeur de la variable *$number*. La valeur est réduite de 1. L résultat est restitué sous la forme d'une nouvelle valeur de $number.
Post-incrémentation	*$number++*	La valeur actuelle de *$number* est d'abord restituée et ensuite augmentée de la vale
Post-décrémentation	*$number–*	La valeur courante de *$number* est d'abord restituée et ensuite diminuée de la valeu

Nous vous présentons à présent des opérations de calcul avec des opérateurs de décrémentation et d'incrémentation. Le premier exemple porte sur la pré-incrémentation. Le script suivant augmente la valeur de la variable *$number* de 1, enregistre la nouvelle valeur dans la variable *$result*, puis restitue sa valeur sous forme de chaîne :

```
<?php
```

$number = 0;

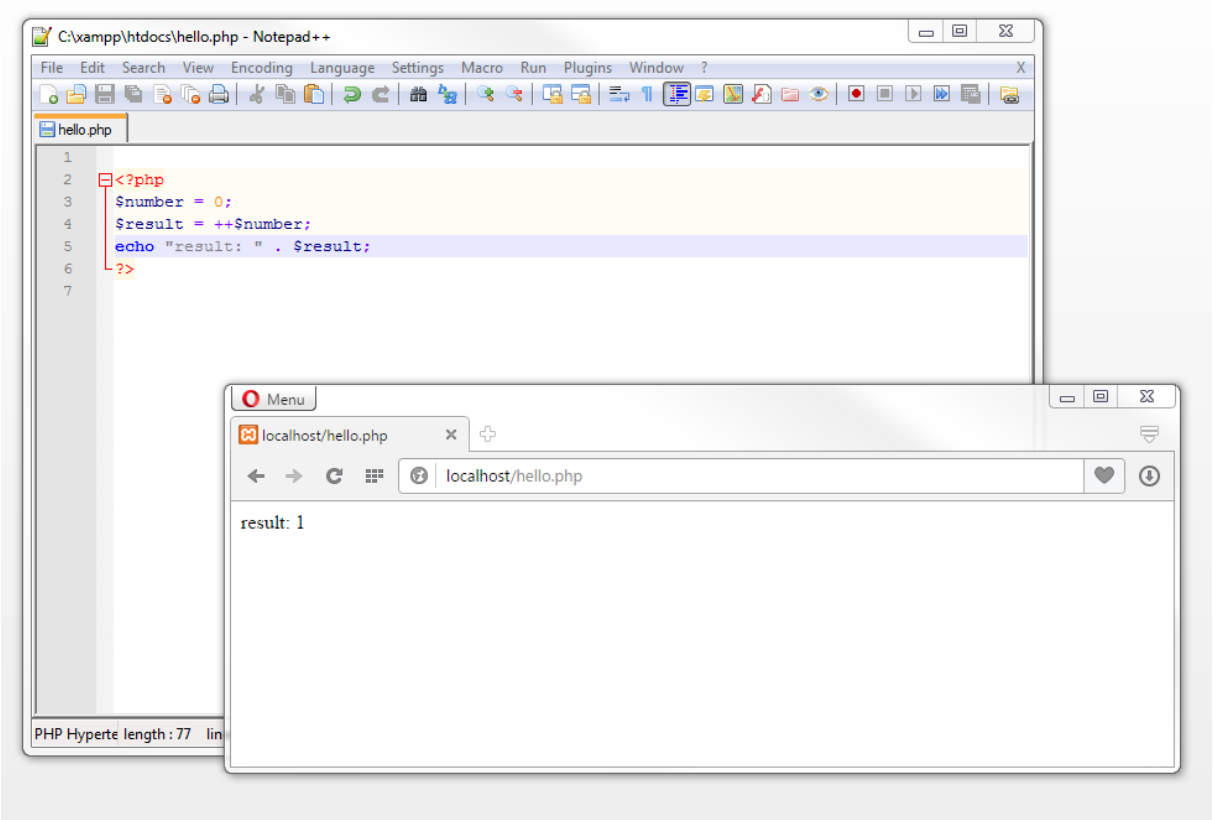
$result = ++$number;

echo "Result: " . $result;

?>

php

Si l'on augmente la valeur 0 de 1, on obtient le résultat 1.



L'opérateur ++ incrémente le chiffre 0, d'où le résultat 1.

Pour calculer le pré-décrément de la variable *$number*, nous utilisons le même script, en remplaçant toutefois l'opérateur pré-incrémental (*++*) par l'opérateur pré-décrémental (–) :

<?php

$number = 0;

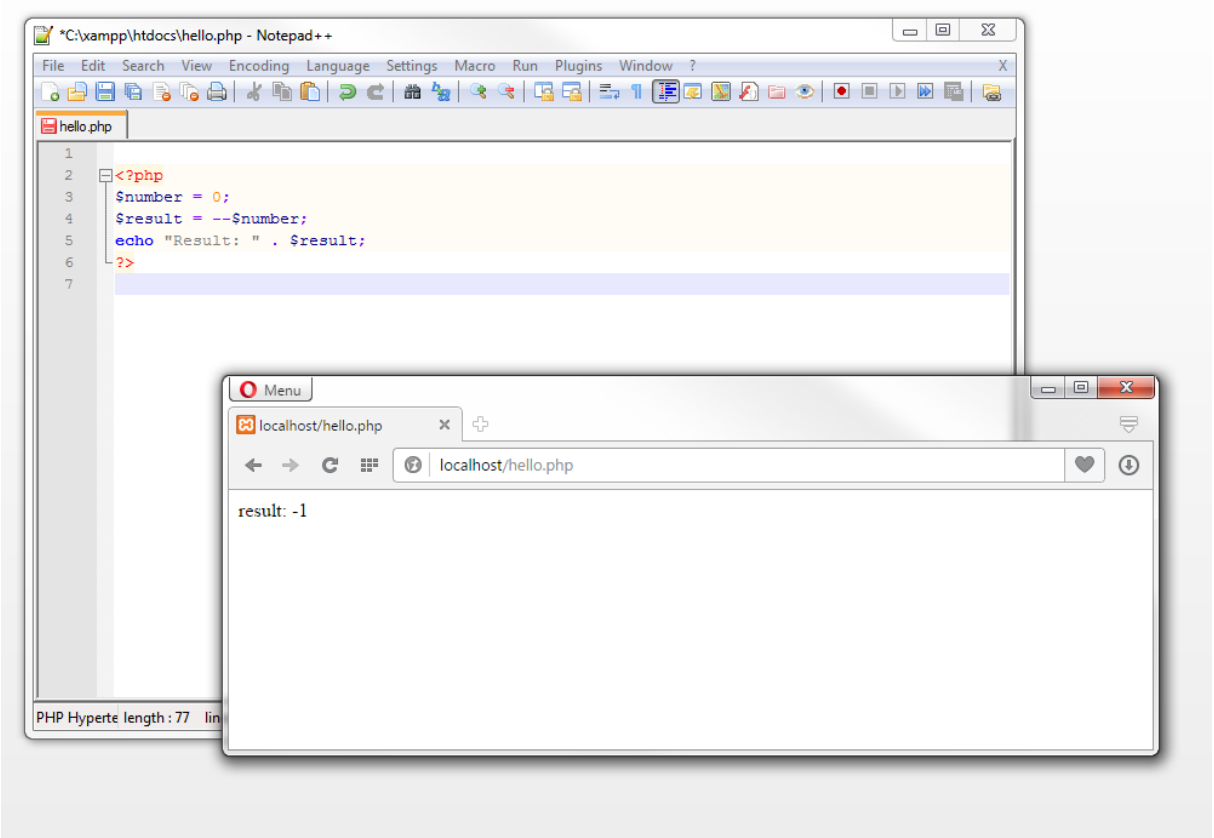
$result = --$number;

echo "Result: " . $result;

?>

php

Nous diminuons ainsi la valeur 0 de la variable *$number* pour obtenir le résultat -1.



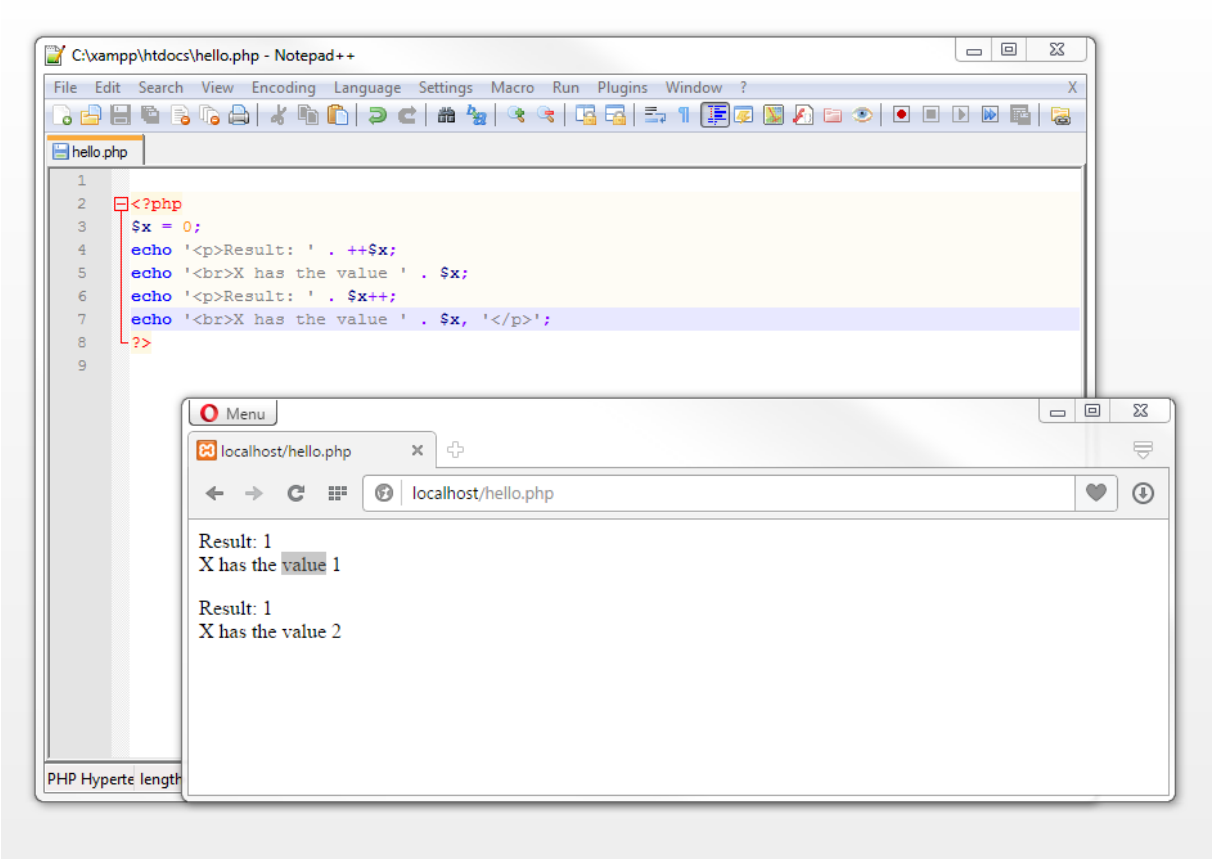
L'opérateur – décrémente le chiffre 0 pour afficher le résultat -1.

Le script suivant indique clairement comment augmenter une valeur avant et après sa publication :

```
<?php

$x = 0;

echo '<p>Result: ' . ++$x;

echo '<br>x has the value ' . $x;

echo '<p>Result: ' . $x++;

echo '<br>x has the value ' . $x, '</p>';

?>
```

php

Dans les deux cas, nous obtenons le même résultat. Dans le cas d'une pré-incrémentation, la valeur *x* est augmentée avant la publication à la ligne 3, et dans le cas d'une post-incrémentation, elle est augmentée après la publication à la ligne 5.



Comparaison de la pré-incrémentation et de la post-incrémentation

Les superglobals $_GET et $_POST

Vous connaissez à présent les bases de PHP, vous savez comment gérer les variables, vous pouvez les concaténer et faire des calculs. Nous vous expliquons maintenant pourquoi les variables constituent un concept central dans la programmation de scripts. L'une des fonctions importantes des langages de script réside dans la capacité d'évaluer la saisie de l'utilisateur et de transférer les valeurs vers un autre script. Pour ce faire, PHP s'appuie sur les **superglobals $_GET et $_POST**, des **variables système** prédéfinies et disponibles dans tous les domaines valides. En tant que tableaux associatifs (arrays),*$_GET et $_POST* stockent un ensemble de variables sous forme de chaînes dans une variable.

Les arrays PHP peuvent être visualisés comme une commode à plusieurs tiroirs, chacun d'entre eux permettant de stocker des données. Afin de vous souvenir ensuite de ce qui se trouve dans chacun de ces tiroirs, étiquetez-les avec un nom de variable. Selon le type de tableau (ou array), il peut s'agir d'un **index ou d'une clé (key)**. Lorsque que vous attribuez un index sous forme de nombre à chaque tiroir dans des tableaux indexés, cela revient à étiqueter les tiroirs d'un array associatif avec une clé sous la forme d'une chaîne (string).

Les superglobals *$_GET et $_POST* contiennent une série de variables sous forme de clés permettant d'accéder aux valeurs associées à ces clés. Voici en détail comment cela fonctionne.

Transfert de données avec $_GET

Le superglobal *$_GET* représente un tableau de variables transmises à un script PHP grâce à une URL.

Si vous vous rendez de temps en temps sur des **blogs**, des**boutiques en ligne** ou des **forums**, vous avez peut-être remarqué que l'on y trouve des URL étranges. On utilise en général une structure qui obéit au schéma suivant :

http://nomdhote/dossier/nomdefichier.php?nomdevariable=valeurdevariable

Pour un blog, cela se traduit ainsi :

http://www.exemple-blog.fr/index.php?id=1

Ce type d'URL peut facilement être décomposé : sur un serveur Web avec le domaine *exemple-blog.fr* se trouve un fichier nommé *index.php*, qui sert à créer un site Web dynamique. Il contient en général du code HTML et PHP, ainsi que des références à des fichiers types (template) et des feuilles de style externes, soit tout le nécessaire pour afficher un site Web.

Il s'agit probablement d'une page Web dynamique, comme l'indique l'ajout après le point d'interrogation (*?*) de *id=1*. Cela s'appelle une chaîne de requête HTTP : elle contient une variable (*id*) et une valeur (*1*) associée à un signe égal (=). Les **paramètres URL** de ce type sont par exemple utilisés pour créer une page Web dynamique, charger du contenu à partir d'une base de données ou sélectionner un template approprié.

Les pages Web dynamiques permettent de **séparer le contenu et la présentation**. L'*index.php* contient certes toutes les informations sur la structure du site Web, mais doit être complété avec du contenu. Ces informations sont généralement stockées dans une **base de données** et peuvent être récupérées à l'aide des paramètres de la chaîne de requête HTTP. Dans notre exemple, l'URL d'*index.php* contient le paramètre *id=1*, qui spécifie quel contenu doit être lu depuis la base de données et chargé dans *index.php*. Pour un blog, cela correspond en général à l'ID d'un article particulier. De cette façon, il est possible de trouver une entrée spécifique dans un forum et un produit spécifique sur une boutique en ligne.

Si une URL contient plusieurs paramètres, ceux-ci sont reliés entre eux grâce à une **esperluette (&)**.

www.exemple-blog.fr/index.php?page=article&id=1

Pour illustrer l'utilisation de *$_GET* avec un exemple de code, il n'est pas forcément nécessaire d'utiliser une base de données. Dans le script suivant, nous utilisons le superglobal *$_GET* pour lire les valeurs des variables *forenameetsurname* d'une chaîne de requête HTTP, et les écrire dans les variables PHP *$variable1 et $variable2* :

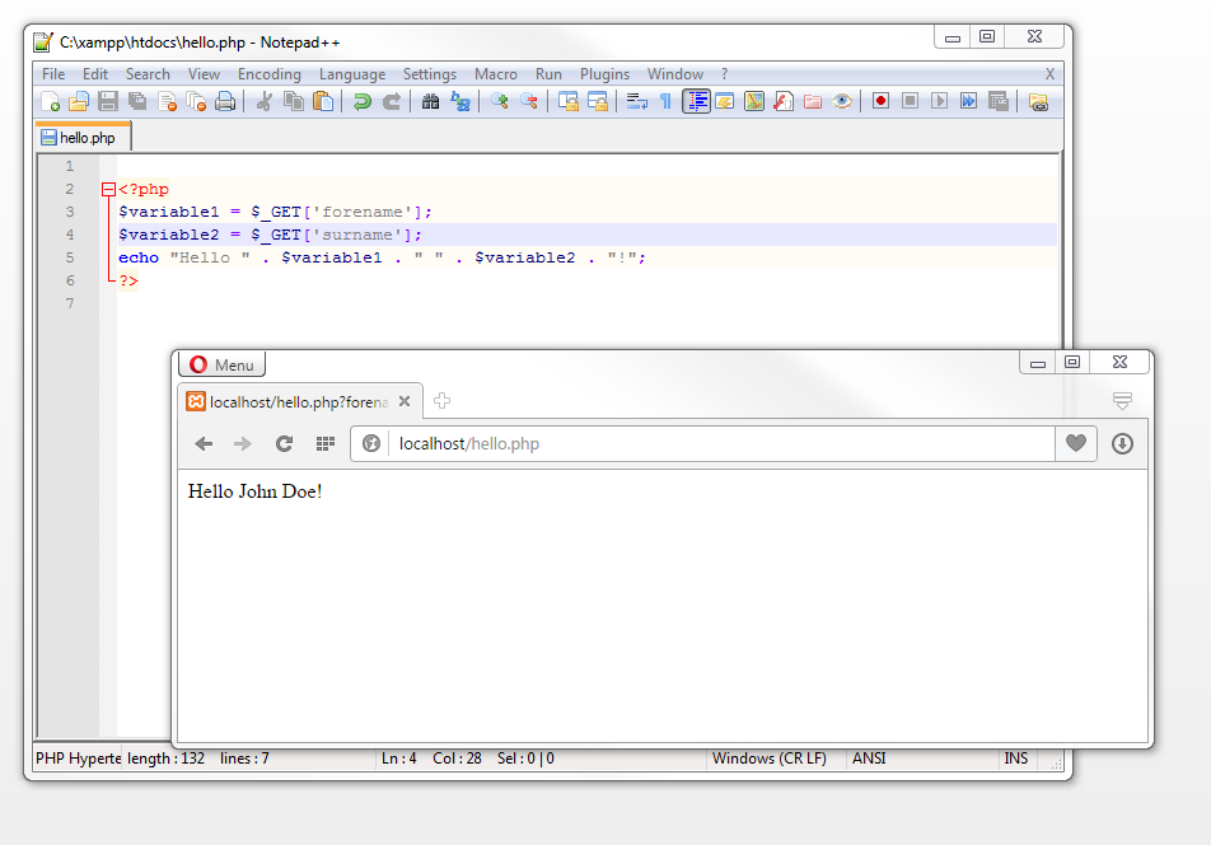
```
<?php

$variable1 = $_GET['forename'];

$variable2 = $_GET['surname'];

echo "Hello " . $variable1 . " " . $variable2 . "!";

?>
```

php

Le script est généré grâce à l'URL suivante :

localhost/hello.php?forename=John&surname=Doe

Nous attribuons donc les paramètres *forename=John*et*surname=Doe*. Comme pour les autres exemples, les valeurs sont publiées grâce à la structure de langage *echo*.



Les paramètres URL transmettent les paires de valeur des variables forename=John et surname=Doe.

Un transfert de données via *$_GET* rend automatiquement les **données visibles dans la barre d'adresse**. Il est donc possible de savoir à tout moment quels sont les paramètres transférés. Ceci présente l'avantage de pouvoir stocker les variables dans des hyperliens. De plus, les utilisateurs peuvent stocker des URL, y compris des chaînes de requêtes HTTP, en tant que signets dans le navigateur.

Cependant, le fait que les paramètres GET soient listés en texte clair dans l'URL rend cette méthode inutilisable pour le transfert de données sensibles, comme les formulaires en ligne. En outre, la quantité de données pouvant être transférée à l'aide de *$_GET* est limitée, car les URL ont une longueur maximale.

Il est possible de contourner ces restrictions grâce à la méthode HTTP POST. Les données ainsi transférées sont disponibles dans le superglobal *$_POST*.

Transfert de données via $_POST

Pendant que les données à transférer sont transmises en tant que paramètres URL avec la méthode GET, avec *$_POST*, les données sont transférées dans le corps d'une requête HTTP. Ceci permet également de transférer de grandes quantités de données d'un script à un autre.

L'une des applications centrales de la méthode HTTP POST réside dans le **transfert des données de formulaire HTML**. Pour mieux comprendre, voici l'exemple d'une inscription à une newsletter.

Créez un nouveau fichier PHP nommé *page1.php* et copiez le bloc de code suivant :

```
<form method="post" action="page2.php" >

Please send the newsletter to: <br />

Forename: <input type="text" name="forename" /><br />

Surname: <input type="text" name="name" /><br />

Email address: <input type="text" name="email" /><br />

<input type="submit" value="Submit Form" />

</form>
```

php

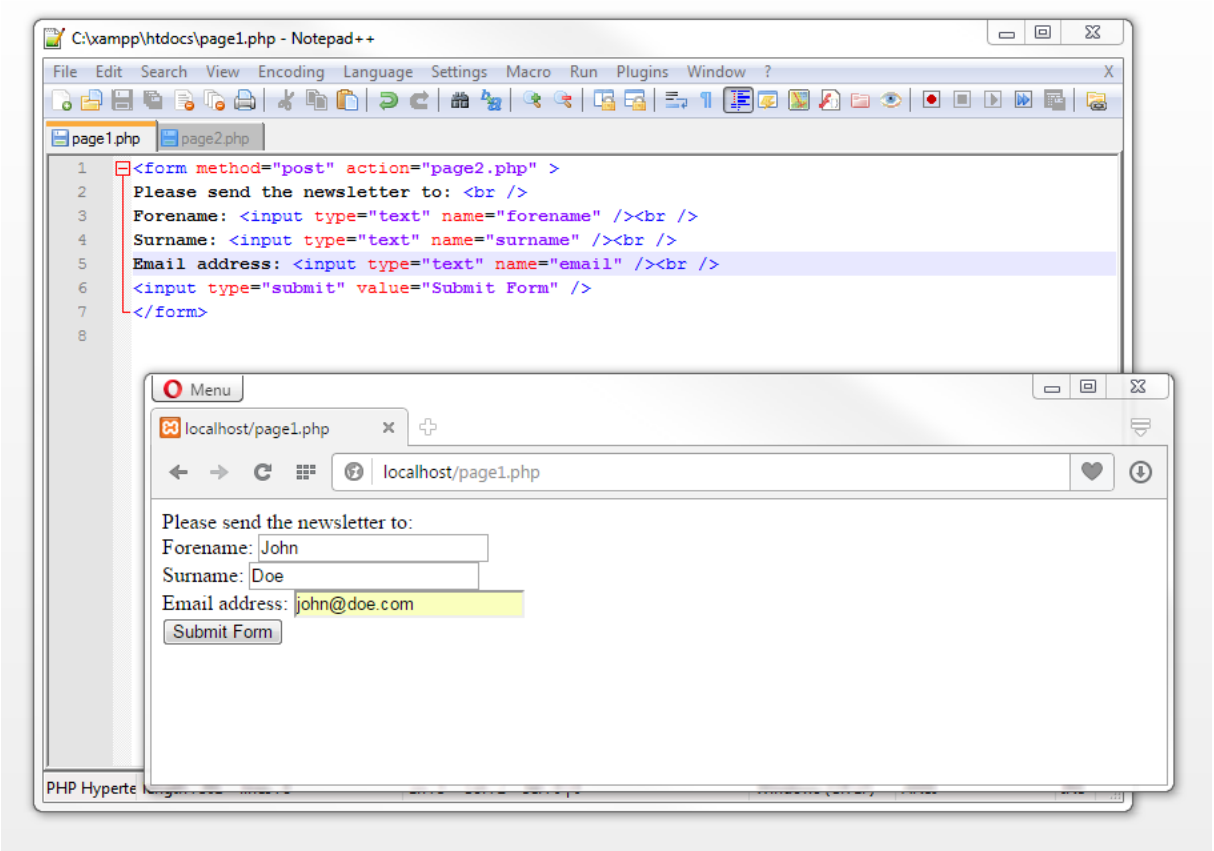
L'élément HTML <form> est utilisé pour créer des formulaires. Il contient deux attributs dans la balise ouvrante : *method*et*action*. L'attribut *method* vous permet de définir la méthode de transfert : dans ce cas, il s'agit de HTTP-POST. L'attribut *action* enregistre l'URL d'un script qui reçoit toutes les données saisies dans les champs suivants. L'exemple montre un **formulaire HTML avec trois éléments de saisie** (*input type="text"*) et un **bouton d'envoi** (*input type="submit"*). Le fichier *page2.php* est défini comme le destinataire des données.

Afin d'illustrer le transfert de données via *$_POST*, nous utilisons pour l'interprétation des données du formulaire un script simple, qui enregistre les valeurs transmises en tant que variables PHP et les publie sous forme de texte. Pour ce faire, créez un fichier *page2.php* et copiez-y le code de programme suivant :

```
<?php

$forename = $_POST["forename"];

$surname = $_POST["surname"];

$email = $_POST["email"];

echo "Hello " . $forename . " " . $surname . ", <br />

Your are registered with the following email address: " . $email . ".";

?>
```

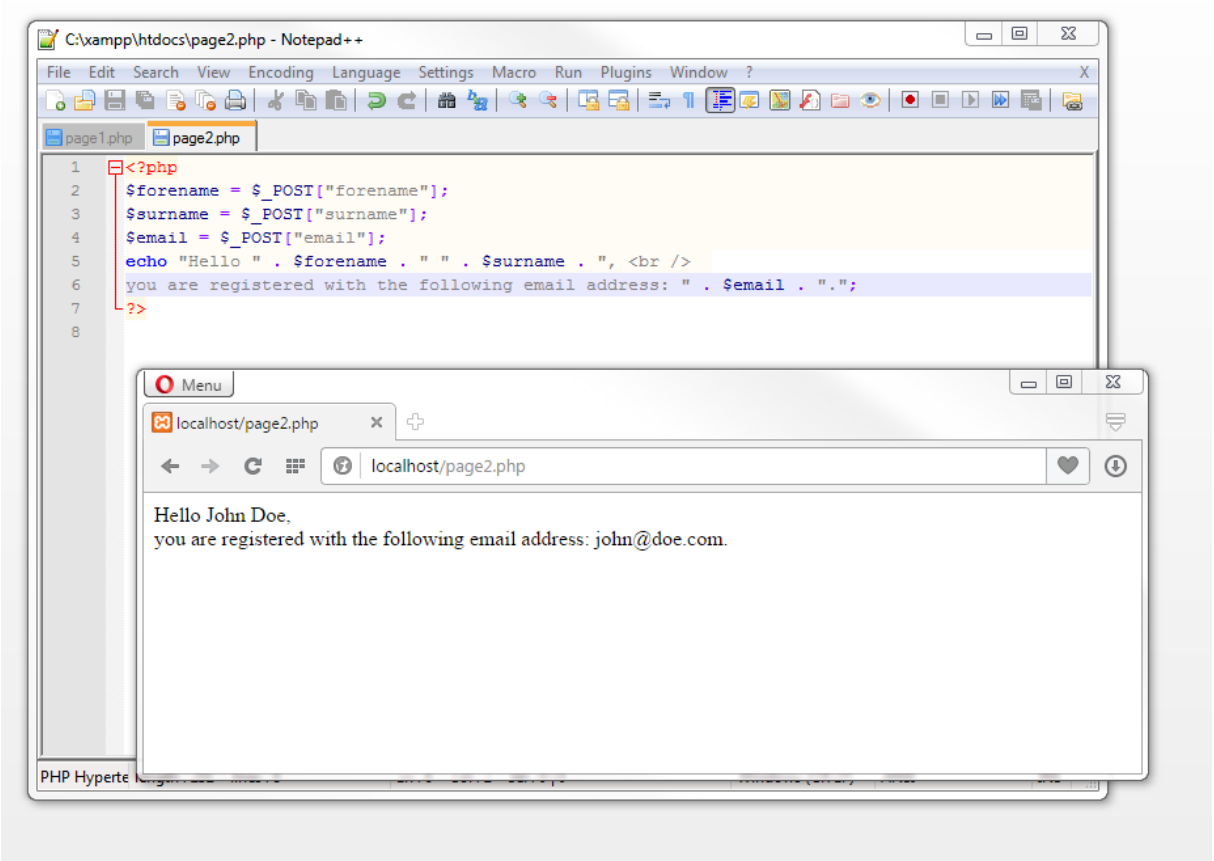
php

Sauvegardez ensuite les deux fichiers PHP dans le dossier *htdocs* de votre serveur de test, puis ouvrez le fichier *page1.php* dans le navigateur Web en saisissant l'URL suivante : http://localhost/page1.php. Votre navigateur affiche maintenant l'interface Web interactive de votre formulaire HTML.



Le formulaire HTTP page1.php rassemble les données utilisateur et les envoie au script avec l'adresse page2.php.

Entrez ensuite les informations d'identification et appuyez sur le bouton Submit pour transférer les variables d'un script à l'autre. Une fois que vous aurez confirmé votre saisie sur *page1.php*, vous serez redirigé vers *page2.php*. La fenêtre du navigateur affiche le résultat de l'exécution du script en fonction des données transmises.



Transfert des données grâce à la méthode HTTP POST et affichage via echo.

Les entrées d'utilisateurs collectées via les champs de saisie de *page1.php* sont récupérées par *page2.php* selon le schéma suivant :

$_POST["Nom du champ de saisie"]

De cette façon, la ligne *$forename = $_POST["forename"]* récupère l'entrée dans le champ de saisie *forename* et l'enregistre dans la variable *$forename*. La variable *$forename* peut ainsi être publiée comme une chaîne de caractères avec *echo*.

L'élément if et les opérateurs comparatifs PHP

Jusqu'à présent, nous avons défini des variables, les avons transférées d'un script à l'autre et les avons publiées sous forme de chaînes. Dans l'étape suivante, vous apprendrez à attribuer certaines *conditions* à l'exécution de fragments de code.

L'**élément de langage if** vous permet d'écrire des scripts de sorte que les instructions ne prennent effet que lorsqu'une condition que vous avez déterminée au préalable est remplie ; il peut par exemple s'agir de la saisie d'un mot de passe correct. En PHP, les conditions sont définies de la façon suivante :

```
<?php

if(expression)

  {

  statement;
```

```
  }
?>
```

php

Ce code se lit comme suit : l'instruction (*statement*) n'est exécutée que si la condition dans l'*expression* est remplie. Une condition est toujours remplie lorsque l'élément *if* affiche le résultat *TRUE* (vrai). Sinon, il est considéré comme *FALSE* (faux) et l'instruction est ignorée.

En règle générale, l'élément *if* vérifie si la valeur d'une variable correspond à ce qui a été défini au préalable dans la condition. Cette **structure de contrôle** est en général mise en œuvre sur la base d'opérateurs comparatifs.

Les opérateurs comparatifs

Les opérateurs comparatifs sont utilisés pour formuler des conditions, afin de mettre deux arguments dans une **relation logique**, qui peuvent être interprétés selon le principe vrai (*TRUE*) ou faux (*FALSE*). Si l'on utilise des opérateurs comparatifs dans les structures de contrôle PHP, ceux-ci sont appliqués à deux variables dans l'*expression* d'un élément *if* :

```
if ($a == $b)
  {
  statement;
  }
```

php

La structure se lit ainsi : si la variable *$a* est égale à *$b*, les instructions définies dans *statement* sont exécutées.

Les **opérateurs comparatifs PHP** sont basés sur le langage de programmation C et leur orthographe diffère en partie des symboles mathématiques classiques. En voici un aperçu dans le tableau ci-dessous :

Opérateur comparatif	Description	Condition
==	Est égal	La condition est remplie lorsque *$a et $b* ont la même valeur.
===	Est identique	La condition est remplie lorsque *$a et $b* ont la même valeur et appartiennent au même type de données. Ceci peut être illustré par un exemple dans lequel un entier (1) est comparé à une chaîne de caractères ("1") : *1 == "1" //TRUE* *1 === "1" //FALSE* Pour les conditions qui requièrent l'égalité des deux variables, il est recommandé de toujours utiliser l'opérateur comparatif === (est identique).
!=	Est différent	La condition est remplie lorsque *$a et $b* n'ont pas la même valeur.
!==	Non identique	La condition est remplie lorsque *$a et $b* n'ont pas la même valeur ou n'appartiennent pas au même type de données.
<	Est inférieur	La condition est remplie lorsque la valeur *$a* est inférieure à la valeur *$b*.

Opérateur comparatif	Description	Condition
>	Est supérieur	La condition est remplie lorsque la valeur *$a* est supérieure à la valeur *$b*.
<=	Est inférieur ou identique	La condition est remplie lorsque la valeur *$a* est inférieure à la valeur *$b* ou que *$a et $b* ont la même valeur.
>=	Est supérieur ou identique	La condition est remplie lorsque la valeur *$a* est supérieure à la valeur *$b* ou que *$a et $b* ont la même valeur.

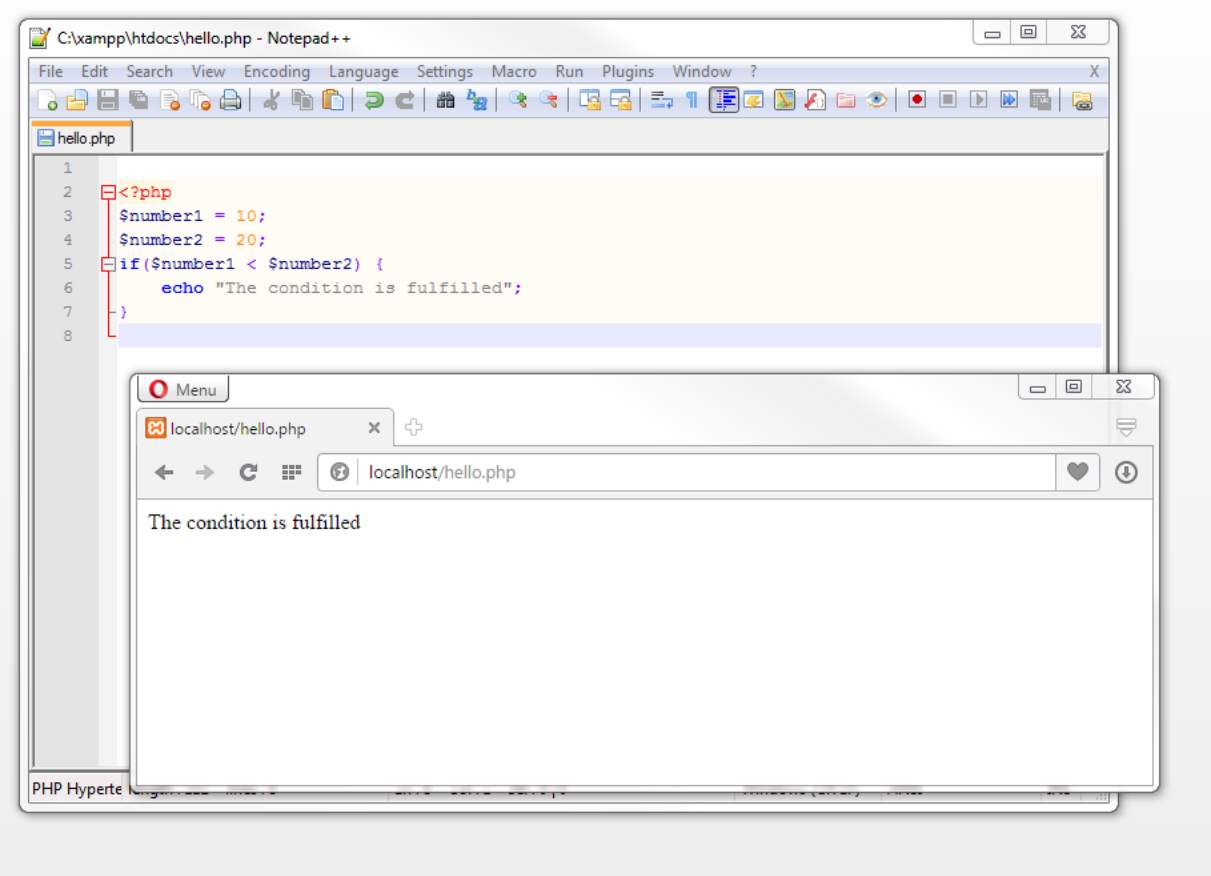
Le script suivant permet de clarifier cette structure de contrôle en comparant deux entiers. C'est l'opérateur comparatif < (Est inférieur) qui est utilisé :

```
<?php

$number1 = 10;

$number2 = 20;

if($number1 < $number2) {

   echo "The condition is fulfilled ";

}
```

php

Nous définissons les variables *$number1 et $number2*, puis leur attribuons les valeurs *10*et*20*. Nous établissons ensuite la condition suivante : si *$number1* est inférieur à *$number2*, la chaîne spécifiée dans l'instruction *echo* doit être affichée.

Le résultat de l'exécution du script contient la réponse : 10 est inférieur à 20 et l'élément *if*renvoie le résultat*TRUE*. La condition est donc remplie (« The condition is fulfilled »).



Si la condition est remplie, l'instruction est exécutée.

Si vous voulez définir des instructions qui sont exécutées lorsqu'une condition n'est pas remplie, ajoutez la structure de langage *else* à la structure de contrôle *if* afin de former une instruction if-else en PHP :

```
<?php

if(condition a)

  {

  Instruction b;

  }

else

  {

  Instruction c

  }

?>
```

php

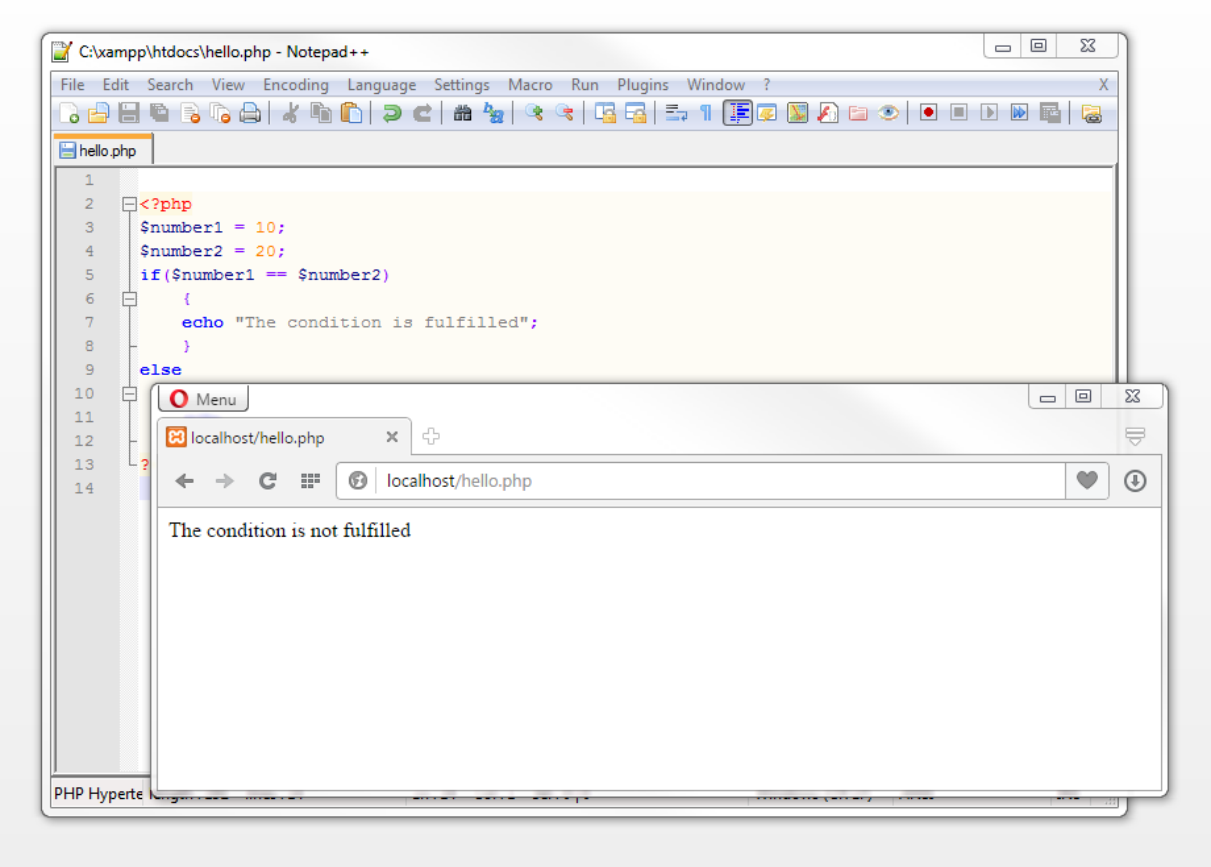
Ce script permet également de vérifier si la *condition a* renvoie un résultat *TRUE* ou *FALSE*. Si la *condition a* est valide (*TRUE*), l'*instruction b* est exécutée. Dans le cas contraire (*FALSE*), le programme passe outre l'*instruction b* et exécute à la place l'*instruction c*.

Complétons notre script avec la structure *else* et remplaçons l'opérateur comparatif < (Est inférieur) par == (Est égal) :

```
<?php

$number1 = 10;

$number2 = 20;

if($number1 == $number2)

  {

  echo "The condition is fulfilled";

  }

else

  {

  echo "The condition is not fulfilled";

  }

?>
```

php

Cette fois, l'élément *if* donne le résultat *FALSE*. La valeur de la variable *$number1* est différente de celle de *$number2*. La condition n'est pas remplie. Cela signifie que la condition n'est pas exécutée selon la structure *if*, mais selon la structure *else*.



Si la condition n'est pas remplie, elle est définie selon l'instruction else.

Note

Si l'exécution d'un fragment de code dépend de l'égalité de deux valeurs, le langage PHP utilise un double signe égal (==). Le signe égal simple (=) n'est utilisé que pour affecter des valeurs aux variables.

La **négation des conditions** se fait par un point d'exclamation (*!*) précédant l'*expression*.

```
<?php

$number1 = 10;

$number2 = 20;

if ($number1 == $number2)

   {

   echo "The numbers are equal.";

   }

if (!($number1 == $number2))

   {

   echo "The numbers are not equal.";

   }
```

?>

php

Cet exemple montre la condition *$number1 == $number2* et sa négation *!($number1 == $number2*), qui correspond à (*$number1 != $number2*).

L'une des applications concrètes de *if*et*else* réside par exemple dans la **requête de mot de passe** basée sur un formulaire HTML. En voici un exemple avec nos fichiers PHP *page1.php*et*page2.php*.

Commencez par ouvrir *page1.php* et entrez-y le code de formulaire suivant :

```
<form action="page2.php" method="post">

Please enter your password: <input type="password" name="password" />

<input type="submit" value="Submit" />

</form>
```

php

La structure correspond au formulaire déjà créé. Mais cette fois, un seul champ de saisie suffit : la requête du mot de passe. Comme précédemment, les saisies de l'utilisateur sont envoyées au script *page2.php*.

Nous l'adaptons avec le code suivant afin que la saisie du mot de passe soit **comparée à un mot de passe mémorisé** :

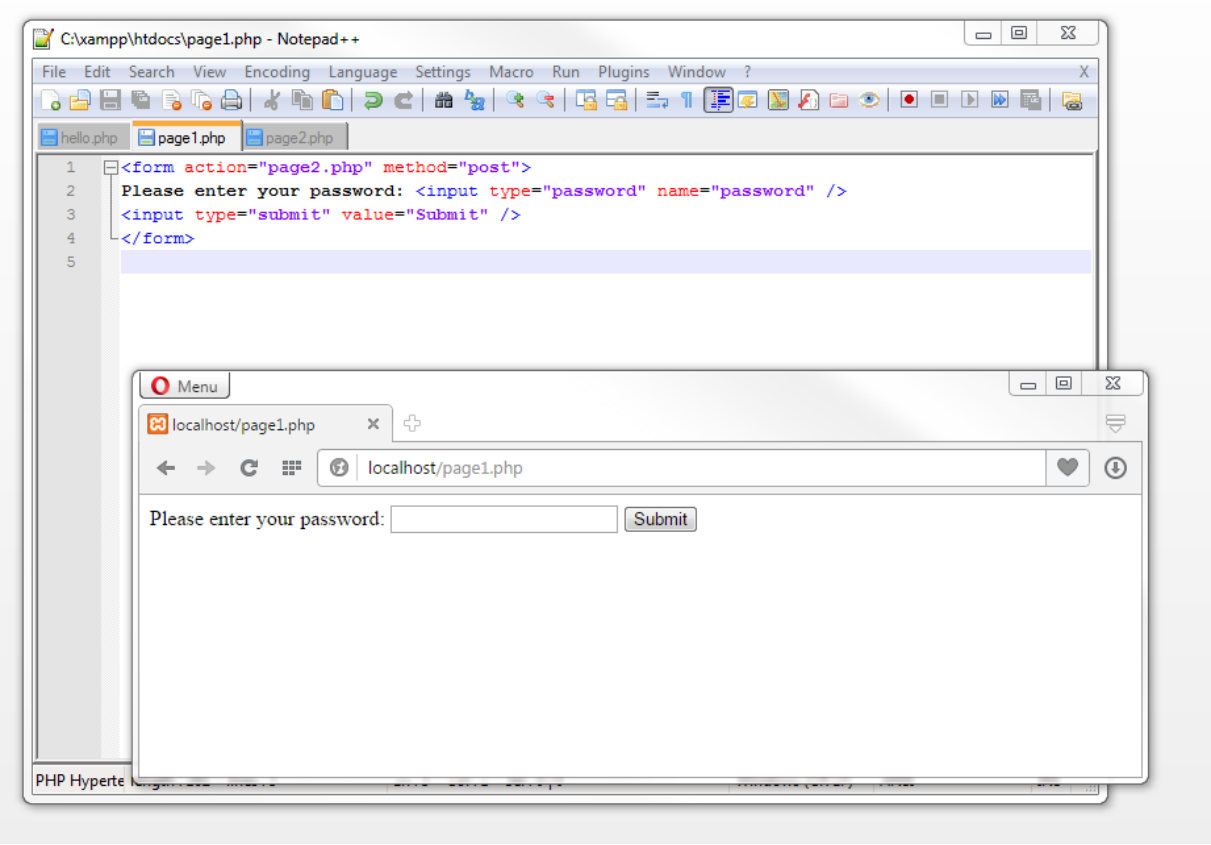
```
<?php

$password = $_POST["password"];

if($password=="qwertz123")

  {

  echo "The password was correct ";

  }

else

  {

  echo "The password was false";

  }

?>
```

php

Le code se lit de cette manière : pour commencer, nous attribuons une valeur à la variable *$password* de la ligne 2, que nous obtenons à l'aide de la méthode HTTP POST. Nous définissons ensuite la structure de contrôle suivante : à la ligne 3, l'élément *if* doit vérifier si la valeur de la variable *$password* est conforme à la chaîne *qwertz123*. Si c'est le cas, nous obtenons la chaîne

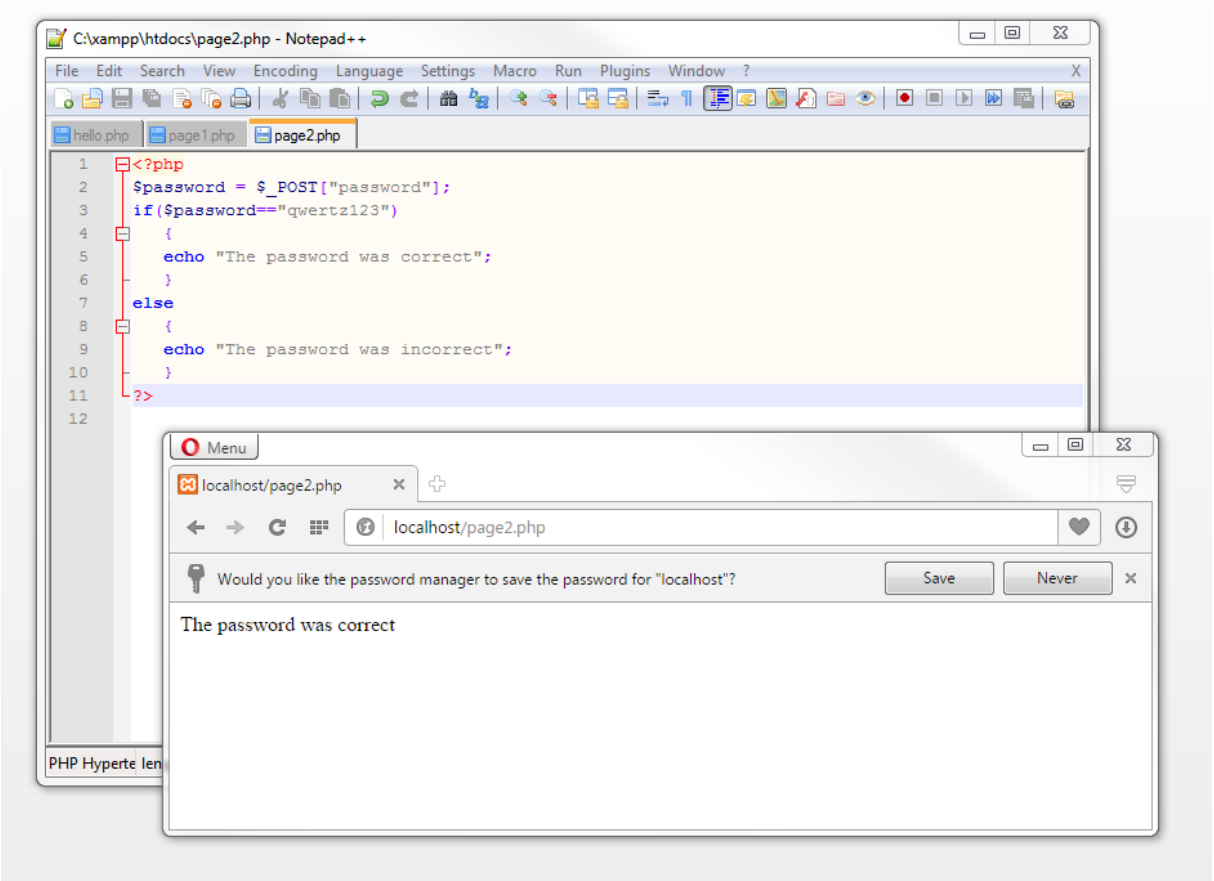
de caractères *The password was correct*. Si *if* donne le résultat *FALSE*, *else* intervient à la ligne 7 et affiche la chaîne de caractères *The password was false*.

Accédons maintenant au script *page2.php* grâce à l'URL http://localhost/page1.php.



Le formulaire HTML vous invite à saisir votre mot de passe.

Le navigateur nous présente l'affichage Web de notre formulaire HTML de requête de mot de passe. Il s'agit alors de saisir le mot de passe *quertz123* défini dans le script *page2.php* et de cliquer sur le bouton *Submit*.



Le script confirme que le mot de passe entré est correct.

Le navigateur Web nous redirige automatiquement vers *page2.php*. En arrière-plan, la structure de contrôle *if* compare notre entrée avec le mot de passe stocké : elle affiche le résultat « *qwertz123 == qwertz123* is *TRUE* » et la chaîne de caractères *The password was correct*.

Testez par vous-même ce qui se passe si vous tapez un mot de passe différent de celui mémorisé dans le champ de saisie.

Les opérateurs logiques

Les conditions que vous définissez à l'aide d'opérateurs comparatifs dans l'*expression* de la structure *if* peuvent être, si nécessaire, liées à d'autres conditions dans la même *expression*. PHP est basé sur les opérateurs logiques **AND**et**OR**.

Connexion forte	Connexion faible	Description
&&	*AND*	Les deux conditions qui sont liées à l'opérateur doivent être vraies (*TRUE*).
||	*OR*	Il faut qu'au moins l'une des conditions liées à l'opérateur soit vraie (*TRUE*).

Pour lier les conditions, PHP confère aux **opérateurs logiques des connexions fortes et faibles**. Si vous n'utilisez qu'une seule des deux orthographes, vous ne remarquerez aucune différence dans la pratique. Si vous combinez les deux orthographes, vous constaterez que *OR et ||* connectent plus étroitement que *AND et OR*. De plus, *AND et &&* sont plus étroitement connectés que *OR et ||*. Cet ordre est comparable à la séquence de classement des opérateurs qui s'applique aux opérateurs

mathématiques (la multiplication est prioritaire sur les additions et soustractions : **permet une connexion plus forte que+*).

L'une des applications concrètes de cette fonctionnalité est la **requête de mot de passe**. En général, les informations d'identification comprennent un mot de passe secret et un nom d'utilisateur. La connexion n'est réussie que si les deux entrées correspondent aux données enregistrées dans le système.

Ouvrons à nouveau notre formulaire de requête de mot de passe *page1.php* et ajoutons un champ de saisie pour le nom d'utilisateur :

```
<form action="page2.php" method="post">

Username: <input type="text" name="username" /><br />

Password: <input type="password" name="password" /><br />

<input type="submit" value="Submit" />

</form>
```

php

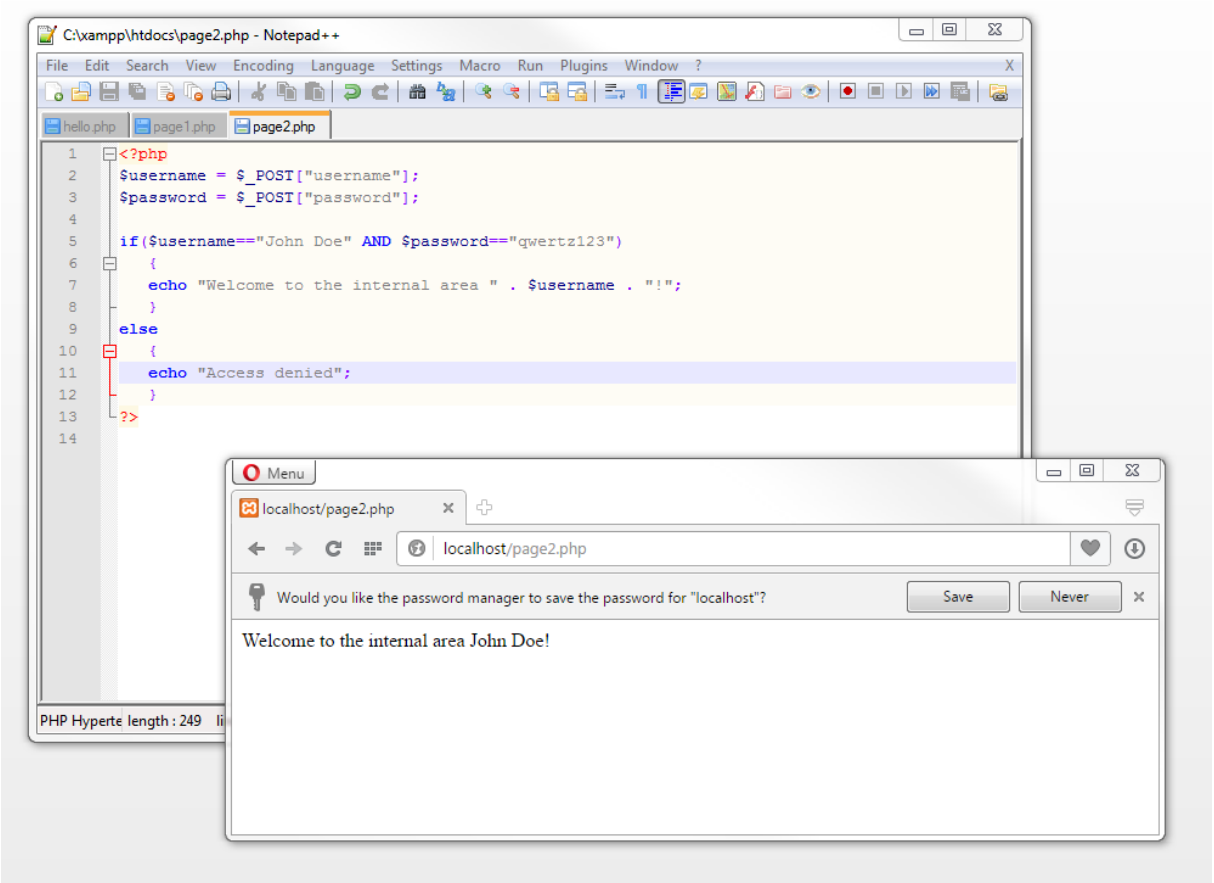
Dans l'étape suivante, nous devons également adapter la structure de contrôle de l'élément *if*. Pour ce faire, nous utilisons l'opérateur logique AND afin de connecter la condition pour la requête du mot de passe à une condition pour la requête du nom d'utilisateur.

```
<?php

$username = $_POST["username"];

$password = $_POST["password"];

if($username=="John Doe" AND $password=="qwertz123")

  {

  echo "Welcome to the internal area" . $username . "!";

  }

else

  {

  echo "Access denied";

  }

?>
```

php

Notre script *page2.php* récupère maintenant les valeurs du nom d'utilisateur (*username*) et du mot de passe (*password*), puis les enregistre dans les variables *$username et $password*. L'*expression* de l'élément *if* contient désormais **deux conditions reliées à l'opérateur logique *AND***. La structure*if*renvoie le résultat*TRUE*seulement si ces deux conditions sont remplies ($username=="John Doe" et $password=="qwertz123").

Puisque l'on obtient le nom d'utilisateur à partir du champ de saisie *username*, il est possible de l'utiliser directement pour la publication de texte via *echo* : *Welcome to the internal area* est suivi de la valeur de la variable *$username*. Si l'une des deux conditions n'est pas remplie, c'est le texte suivant qui s'affiche : *Access denied*.



Ce n'est que lorsque les deux conditions sont remplies que le script confirme que le mot de passe tapé est correct.

Les opérateurs logiques peuvent être combinés à loisir. La règle suivante s'applique alors : l'opérateur *AND* est prioritaire par rapport à *OR*. De même que pour les équations mathématiques, vous pouvez également utiliser les parenthèses en PHP pour influencer cet ordre de priorité.

Les boucles (while, for)

Il est parfois nécessaire qu'un script passe plusieurs fois à travers une certaine section de code avant que le reste du code ne soit exécuté. Pour ce faire, les langages de programmation utilisent le concept de boucles, également appelées boucles PHP. Il existe trois **types de boucles** :

- La boucle *while*
- La boucle *do-while*
- La boucle *for*

La boucle while

Il s'agit du type de boucle le plus simple proposé par PHP. La **construction de base** correspond au schéma suivant :

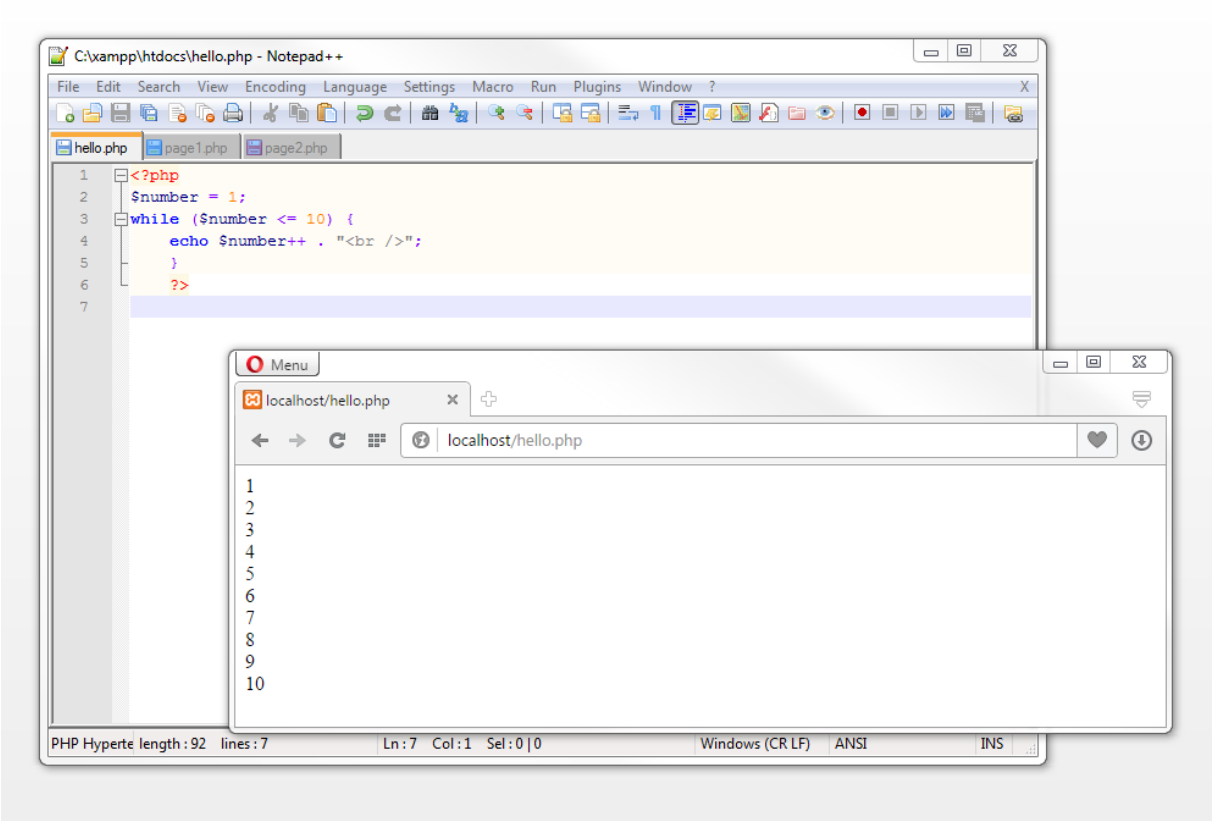
```
while (Condition)
  {
  Étapes de la boucle et autres instructions
  }
```

php

La boucle *while* ordonne à PHP d'exécuter les instructions secondaires tant que la condition *while* est remplie. L'interprète PHP vérifie donc la condition au début de chaque passage de boucle. L'exécution du code secondaire ne s'interrompt que lorsque la condition *while* n'est plus remplie. Ce principe peut être illustré par un simple script de calcul :

```
<?php
$number = 1;
while ($number <= 10) {
  echo $number++ . "<br />";
  }
?>
```

Dans la section « Calculer avec des variables », nous avons introduit le principe de l'incrément. Dans le script suivant, nous l'utilisons à nouveau, mais cette fois-ci avec un opérateur post-incrément pour augmenter de 1 la valeur de la variable entière *$number* à chaque passage de boucle après la publication de texte via *echo*. Comme condition pour la boucle *while*, nous définissons : *$number* supérieur ou égal à 10. L'instruction *echo* est donc répétée jusqu'à ce que *$number* ait atteint une valeur supérieure à 10.



Le script incrémente la variable *$number* jusqu'à ce qu'elle atteigne une valeur supérieure à 10.

Le résultat de l'exécution du script est une chaîne qui renvoie la valeur de la variable *$number* pour chaque passage de boucle avant qu'elle ne soit incrémentée. Le script compte donc de 1 à 10 et met fin à l'exécution du code dès que la condition *while* n'est plus remplie.

La boucle do-while

La structure de la boucle *do-while* est similaire à celle de la boucle *while*. La seule différence est que la condition n'est pas vérifiée au début de chaque passage de boucle, mais uniquement à la fin.
Le **schéma de base d'une boucle *do-while*** correspond à l'exemple suivant :

do {

Étapes de la boucle et autres instructions

}

while (Condition)

php

Programmé comme une boucle *do-while*, le script précédent ressemblerait à ceci :

<?php

$number = 1;

do {

echo $number++ . "<br />";

}

```
while ($number <= 10);

?>
```

php

Dans ce cas, le résultat reste le même. La particularité de la boucle *do-while* est qu'elle est parcourue au moins une fois, même si la condition n'est pas remplie lors d'un passage de boucle.

La boucle *for*

Dans l'ensemble, une boucle *for* au sein d'un script PHP présente les mêmes fonctionnalités qu'une boucle *while*. Cependant, la **valeur de départ, la condition et l'instruction sont écrites sur une seule ligne** au lieu d'être réparties sur trois lignes ou plus. La structure de base de la boucle *for* correspond au schéma suivant :

```
for (valeur de départ; condition; étape de la boucle)

   Instructions
```

php

Avec une boucle *for*, l'exemple précédent pourrait donc être rédigé comme suit :

```
<?php

for($number= 1; $number<= 10; $number++) {

   echo $number . "<br /> ";

}

?>
```

php

Tout d'abord, la valeur *1* est définie pour la variable *$number*. PHP vérifie ensuite si la condition *$number <=1* est remplie. Si c'est le cas, la boucle continue et les instructions situées sous la boucle sont exécutées (il s'agit ici de l'instruction *echo*). Vient ensuite l'exécution de l'étape de boucle. Dans ce cas, peu importe que l'on choisisse la pré-incrémentation ou la post-incrémentation, car cette instruction est toujours exécutée avant la publication. Une fois l'étape de boucle terminée, le passage suivant de la boucle peut débuter.

La valeur de départ, la condition et l'étape de boucle sont des **éléments optionnels** d'une boucle *for*. En théorie, il est même possible de créer des boucles vides, bien que ce soit redondant.

C'est à vous de décider si vous rédigez vos scripts PHP avec une boucle *for* ou while. Un argument plaide toutefois en faveur des boucles for: leur utilisation offre en effet une meilleure vue d'ensemble des données générales de la boucle. Cela réduit le risque de rédiger accidentellement une boucle infinie qui s'exécute jusqu'à ce que la mémoire de l'interprète soit pleine. Ceci peut arriver avec l'exemple précédent si l'on oublie d'augmenter la valeur de la *variable $number*.

Cependant, si la boucle doit être exécutée au moins une fois, quelle que soit la condition, la boucle *do-while* est celle qui convient le mieux.

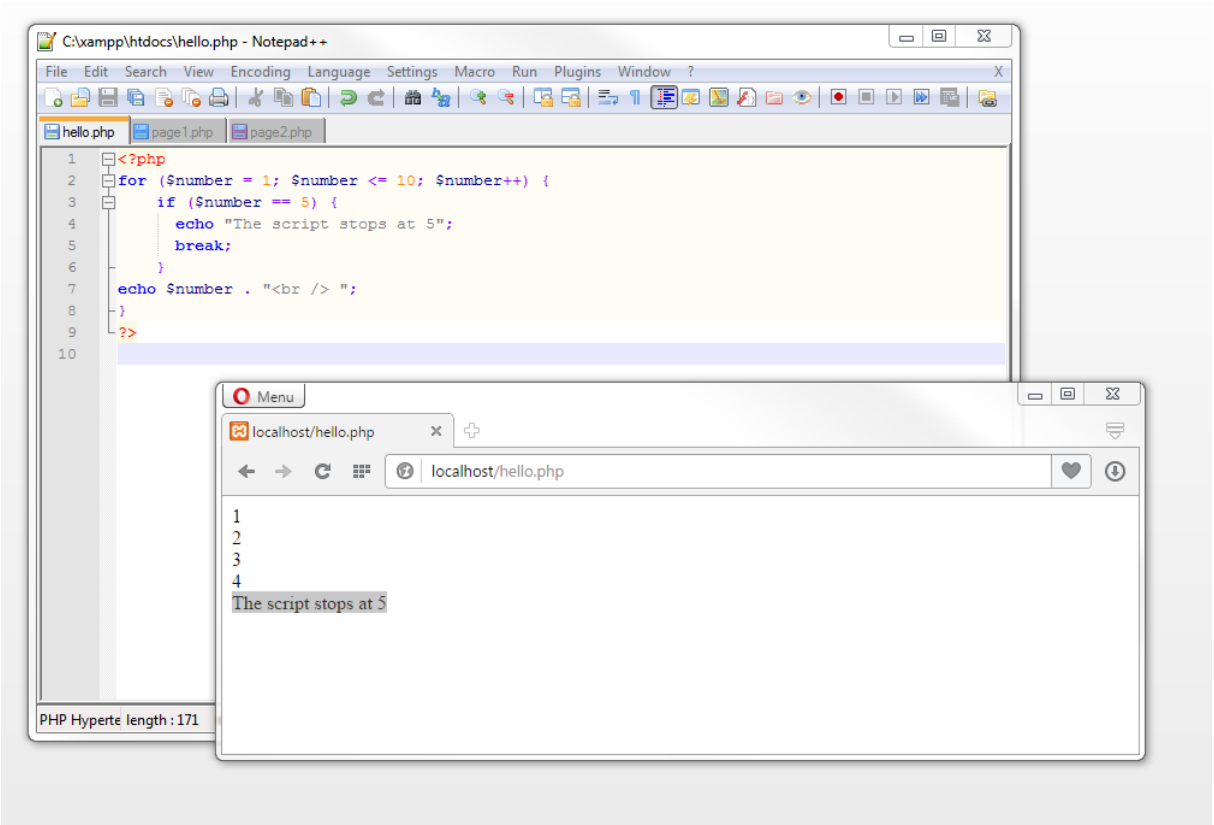
Les instructions break et continue

Le déroulement d'une boucle *while*, *do-while* ou *for* peut être influencé par les instructions *break*et*continue*. Utilisez *break* pour interrompre une boucle à n'importe quel moment, et *continue* pour sauter le passage d'une boucle. Les deux instructions sont liées à une condition par l'intermédiaire de *if*. L'exemple suivant montre notre script de calcul avec *break* :

```
<?php
for ($number = 1; $number <= 10; $number++) {
  if ($number == 5) {
    echo "The script stops at 5";
    break;
  }
echo $number . "<br /> ";
}
?>
```

php

Dans la boucle *for*, nous avons défini que la valeur de la variable *$number* doit être incrémentée de 1 à partir d'une valeur de départ de 1 dans chaque boucle, jusqu'à ce que la variable atteigne 10. Cette **boucle est alors arrêtée plus tôt que prévu** avec l'instruction *break* dès que *$number* atteint 5. La structure de langage *echo* ne produit donc que les nombres de 1 à 4.



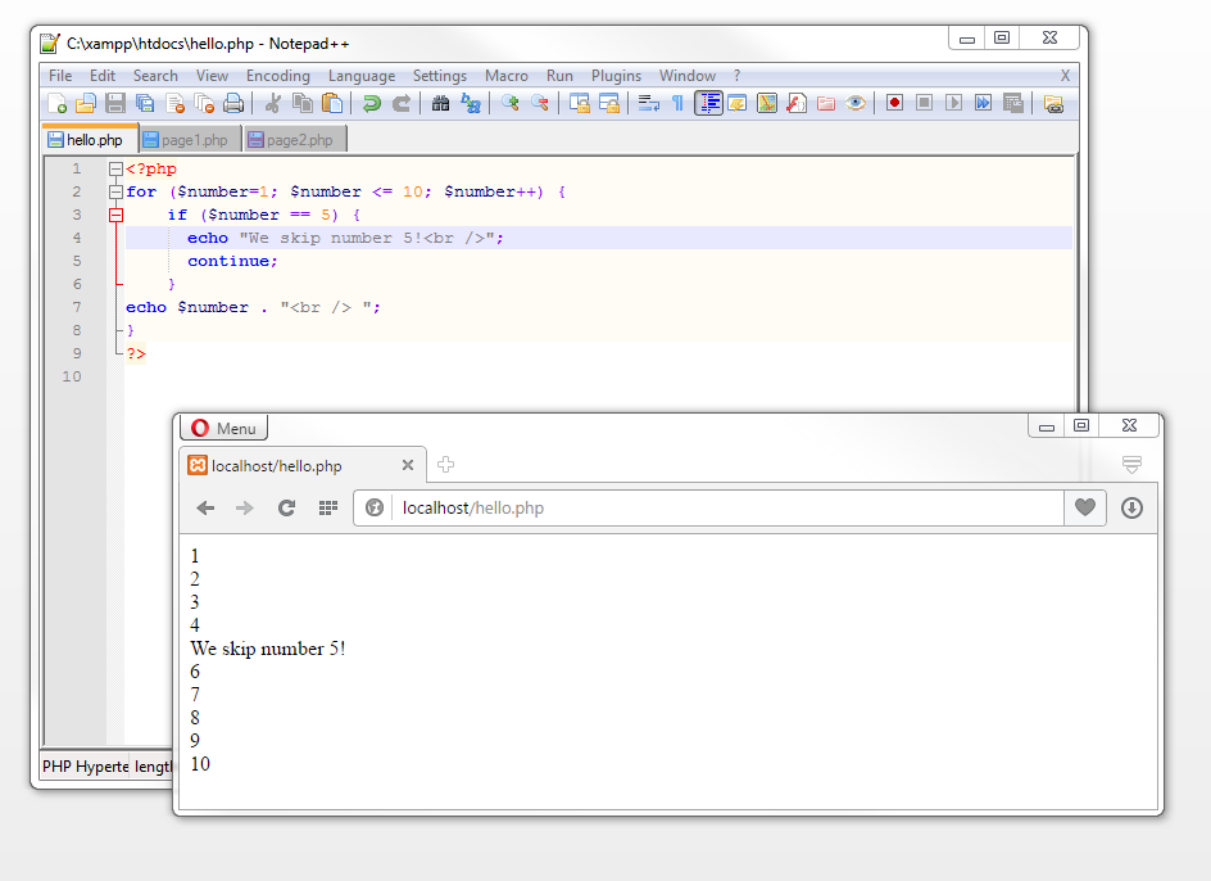
L'instruction break interrompt la boucle dès que la condition if est remplie.

Si nous souhaitons juste **sauter le cinquième tour** sans pour autant interrompre l'ensemble de la boucle, nous devons remplacer l'instruction *break* par *continue*.

```
<?php
for ($number=1; $number <= 10; $number++) {
   if ($number == 5) {
     echo "We skip number 5!<br />";
     continue;
   }
echo $number . "<br /> ";
}
?>
```

php

À la place du chiffre 5, PHP affiche la ligne de texte *We skip number 5!* définie sous *if*.



La boucle for est interrompue grâce à l'instruction continue.

Les opérations sur des fichiers

Le contenu Web dynamique est basé sur une séparation du contenu et de la présentation. Les langages de script tels que PHP offrent donc différentes fonctionnalités qui permettent de charger

du **contenu à partir de sources de données externes** dans des fichiers types (template). Dans la pratique, ces sources de données consistent essentiellement en des bases de données gérées par des systèmes tels que MySQL. Retrouvez les instructions correspondantes dans notre tutoriel MySQL.

Il est également possible d'**intégrer des données issues de fichiers**. Dans l'exemple suivant, nous expliquons comment lire des fichiers sous forme de chaînes dans un script PHP et comment stocker les publications de texte de votre script dans des fichiers.

Lire des fichiers

PHP offre diverses fonctionnalités pour lire le contenu d'un fichier, en particulier *file()etfile_get_contents()*. Si la fonction *file_get_contents()* lit tout le contenu d'un fichier dans une chaîne de caractères, la fonction *file()* en sauvegarde le contenu sous forme de tableau. Chaque élément du tableau correspond à une ligne du fichier. Avec *file()*, il est donc plus facile de modifier individuellement chaque ligne.

Nous présentons les opérations sur fichier PHP sur le fichier texte *example.txt*, que nous stockons dans le dossier *htdocs* de notre serveur de test. Le fichier contient quatre lignes de texte à titre d'exemple :

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Aenean commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Donec quam felis, ultricies nec, pellentesque eu, pretium quis, sem.

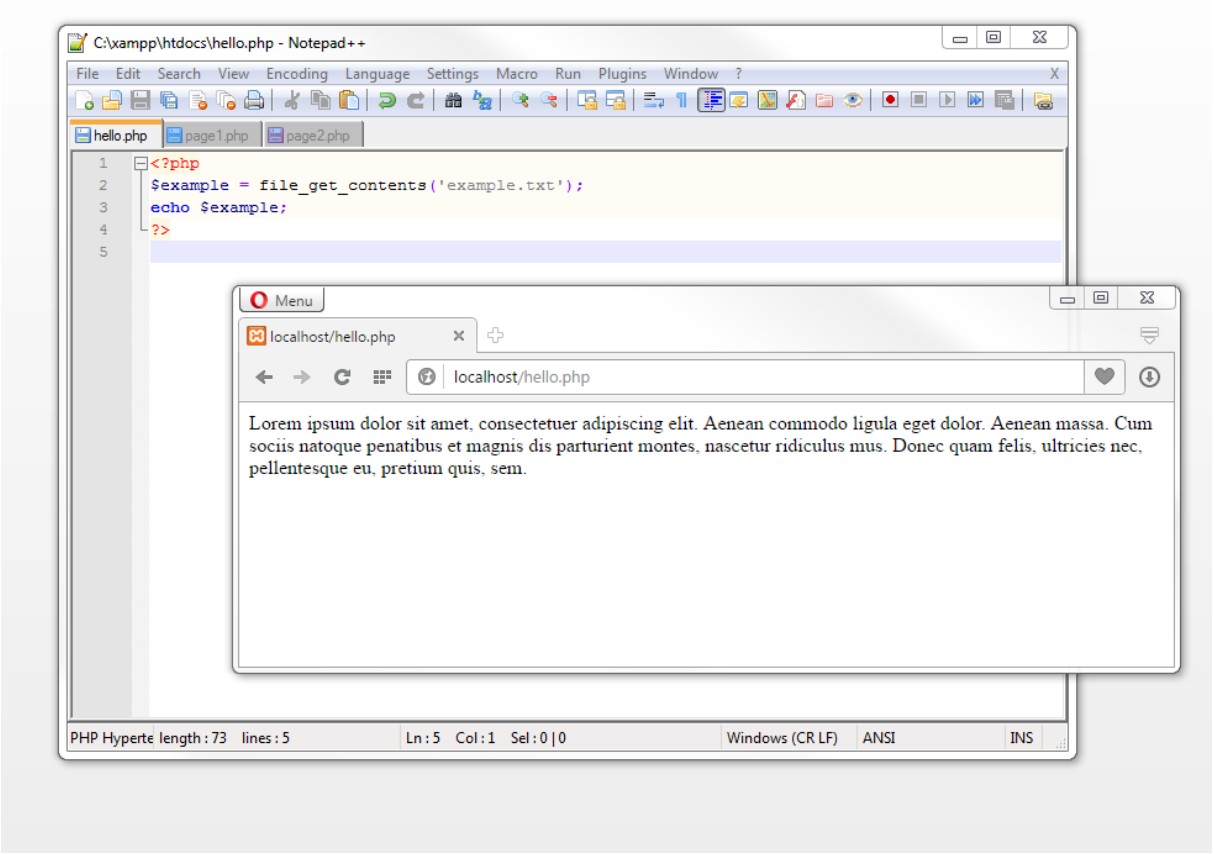
Pour commencer, nous importons l'intégralité du fichier sous forme de chaîne de caractères. Pour ce faire, il est nécessaire d'attribuer le nom du fichier correspondant en tant que paramètre à la fonction *file_get_contents()*. Ceci fonctionne selon le schéma suivant :

```
file_get_contents('example.txt')
```

Nous pouvons désormais travailler avec la chaîne importée. Par exemple, nous pouvons l'assigner à une variable et la modifier sous forme de texte dans le navigateur Web :

```
<?php

$example = file_get_contents('example.txt');

echo $example;

?>
```

php



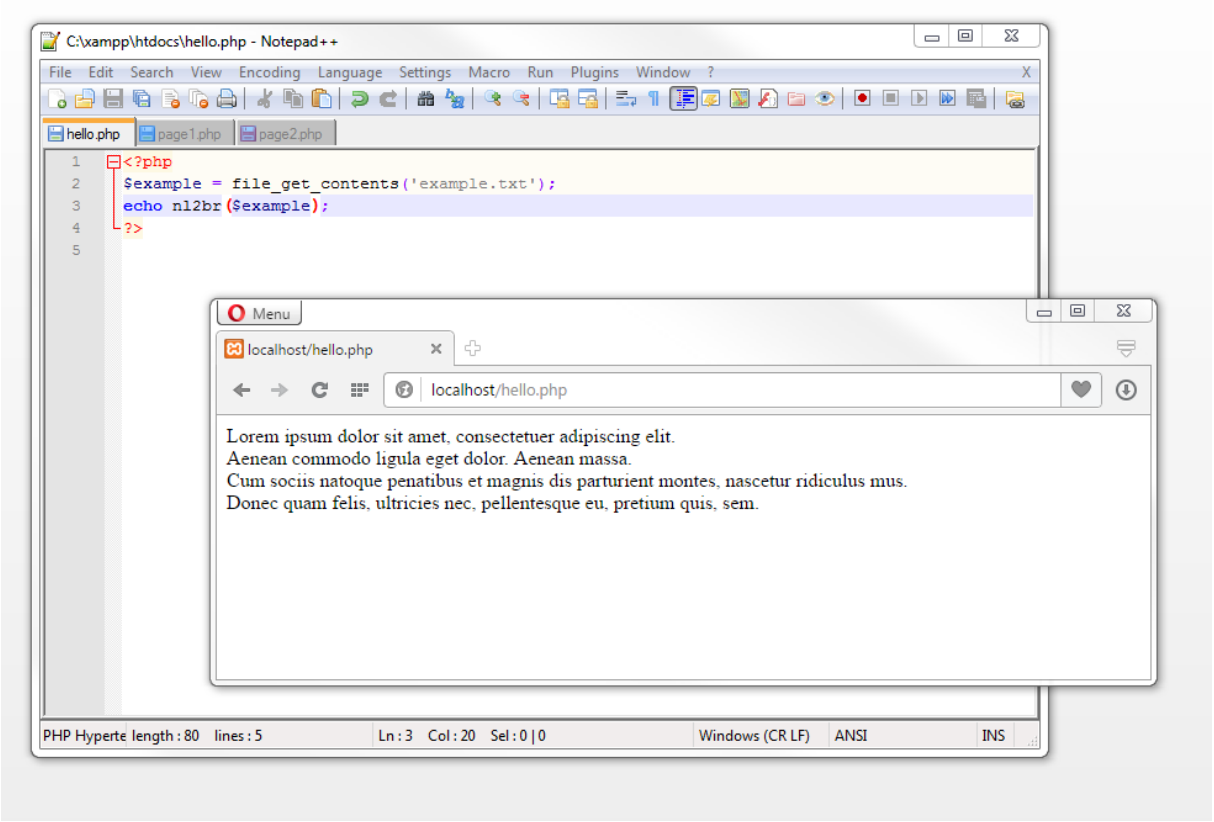
Le fichier lu est affiché dans le navigateur.

Dans l'affichage du navigateur, nous constatons que la chaîne de texte s'affiche sans paragraphes. Les sauts de ligne du fichier d'origine ne sont pas visibles. Ceci s'explique par le fait que le navigateur Web interprète la sortie de texte du script comme du code HTML. Les alinéas définis dans les éditeurs de texte sont donc perdus lors du processus. Si vous souhaitez conserver la structure d'origine, il existe plusieurs options. Vous pouvez ajouter manuellement un saut de ligne (*br*) au code HTML dans le fichier source en utilisant Rechercher et remplacer, paramétrer <pre> pour définir le contenu du fichier, assigner la propriété CSS *white-space: pre-wrap* à la zone ou utiliser la fonction *nl2br()* pour indiquer à PHP que les sauts de ligne (*new lines*) doivent être automatiquement convertis en saut de ligne HTML (*breaks*). Cette fonction s'utilise comme suit :

```
<?php

$example = file_get_contents('example.txt');

echo nl2br($example);

?>
```

php

Si vous utilisez la structure de langage *echo* en combinaison avec *nl2br()*, PHP insère un saut de ligne HTML avant chaque nouvelle ligne.



La fonction nl2br() aide à structurer les données.

Si vous souhaitez **éditer individuellement les lignes d'un fichier**, utilisez la fonction *file()*. Elle permet en effet de lire un fichier, de numéroter toutes les lignes commençant par 0 et d'enregistrer leur contenu comme éléments d'un tableau. Appliquée à notre exemple, cette fonction se présente comme suit :

[0] = *Lorem ipsum dolor sit amet, consectetuer adipiscing elit*

[1] = *Aenean commodo ligula eget dolor. Aenean massa.*

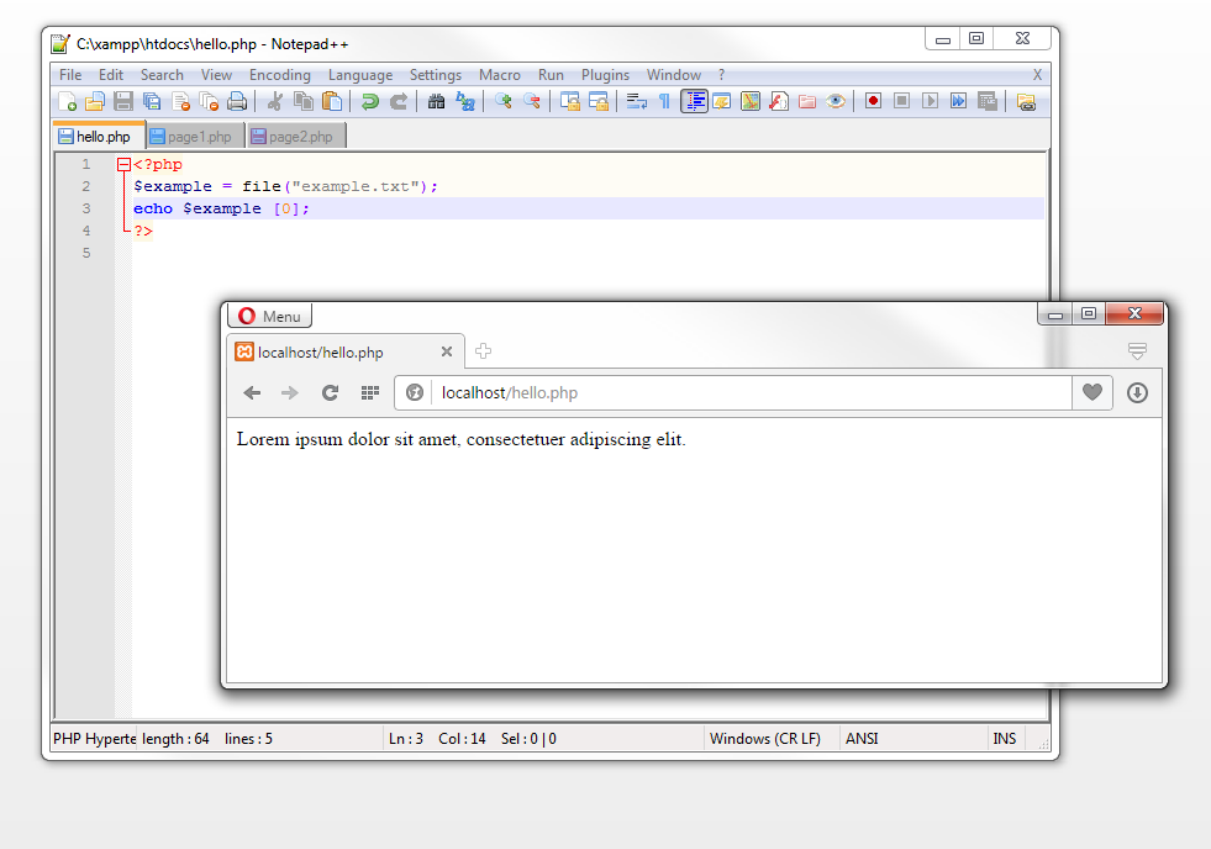
[2] = *Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus.*

[3] = *Donec quam felis, ultricies nec, pellentesque eu, pretium quis, sem*

Pour éditer les contenus respectifs avec *echo*, il vous suffit de spécifier le numéro de ligne que vous souhaitez modifier. Par exemple, le script suivant affiche seulement la première ligne du fichier example.txt sur le navigateur :

```
<?php

$example = file("example.txt");

echo $example [0];

?>
```

php



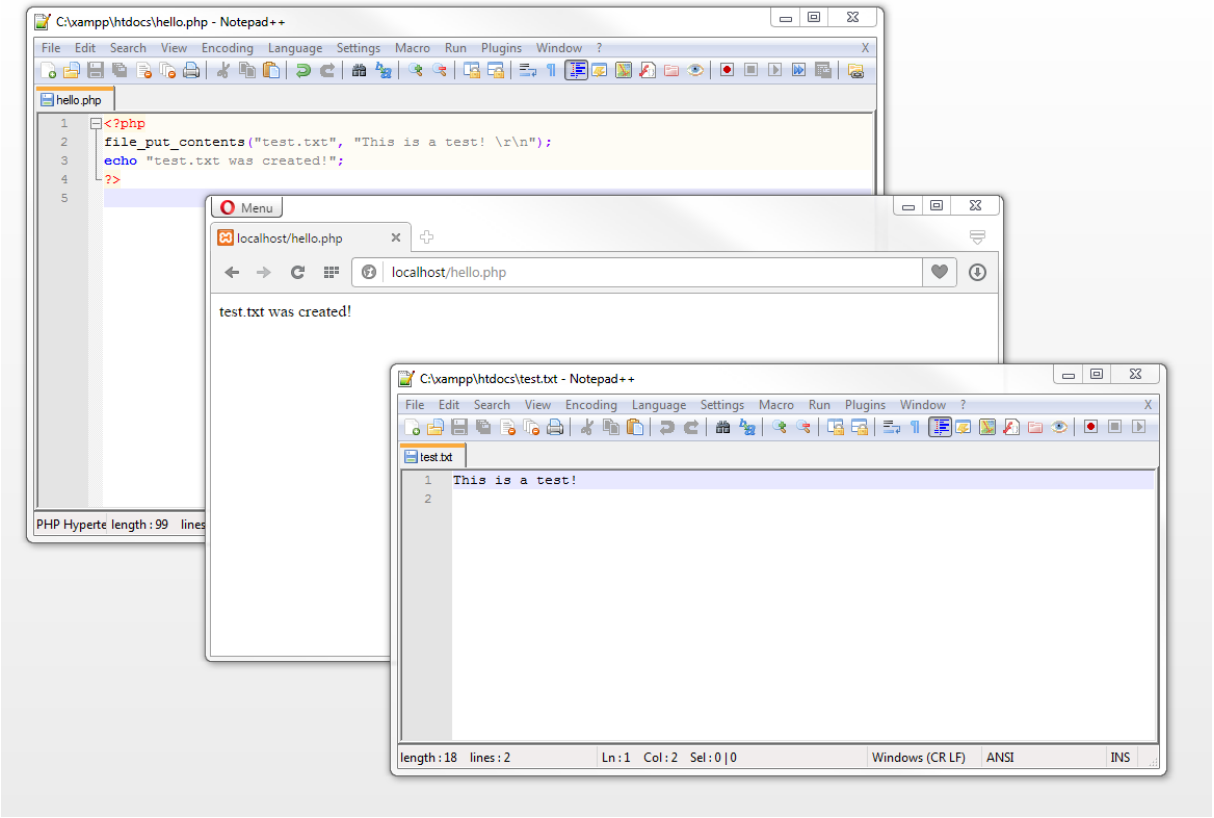
Sélectionnez l'élément du tableau que vous souhaitez modifier.

Écrire dans des fichiers

PHP n'est pas seulement capable de lire les fichiers. Le langage de script vous permet également de créer des fichiers et de les décrire avec du contenu. Pour ce faire, nous utilisons la fonction PHP *file_put_contents()*. Ceci présuppose **deux paramètres** : d'une part le nom du fichier à créer ou à mettre à jour, d'autre part les données sous forme de chaîne ou de tableau. Le script suivant crée le fichier test.txt et écrit la chaîne de caractères *This is a test!* à la première ligne. L'ajout de \r\n provoque un saut de ligne dans le fichier cible.

```
<?php

file_put_contents("test.txt", "This is a test! \r\n");

echo "test.txt was created!";

?>
```

php



Le script PHP écrit la ligne de texte « This is a test! » dans le fichier test.txt.

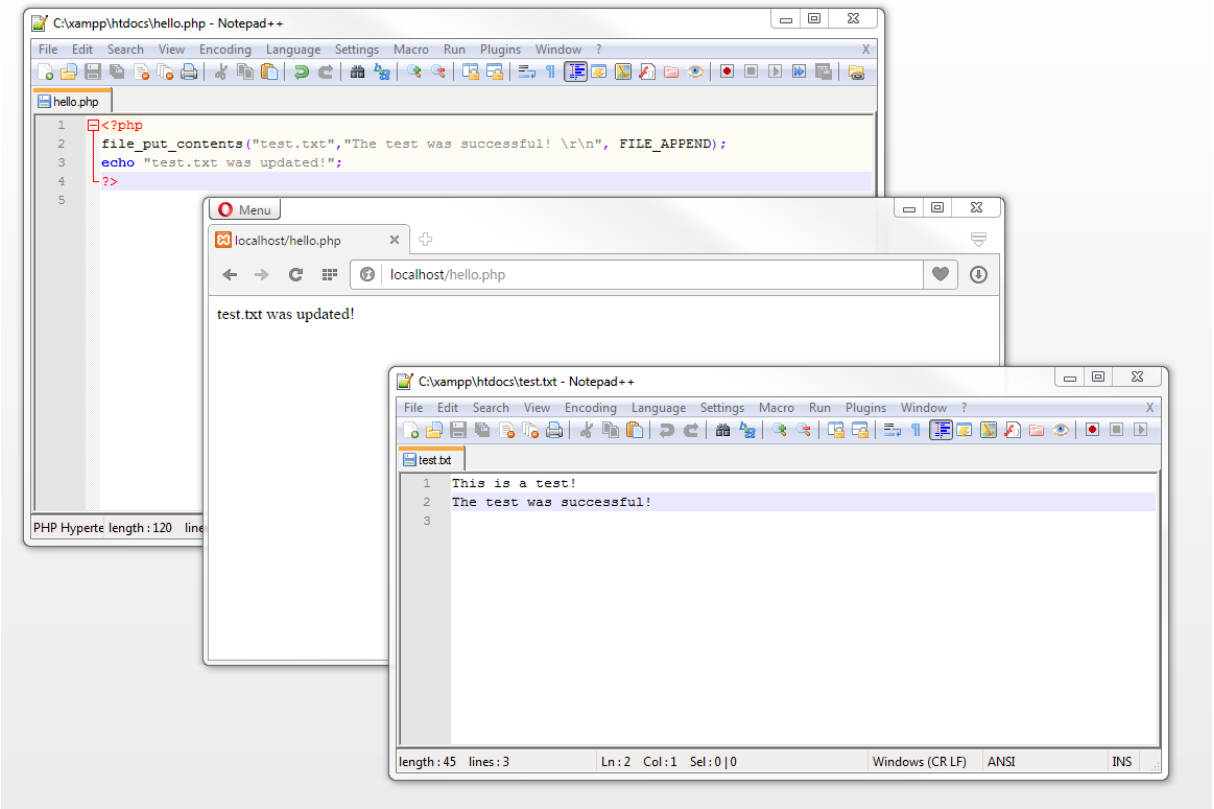
Dans la mesure où *file_put_contents* ne fournit pas d'affichage pour le navigateur nous ajoutons une instruction *echo* qui indique l'action à exécuter.

S'il y a déjà un fichier du même nom dans le dossier cible, celui-ci sera écrasé. Pour éviter cette opération, réglez le paramètre *FILE_APPEND* de la façon suivante :

```
<?php

file_put_contents("test.txt","The test was successul! \r\n", FILE_APPEND);

echo "test.txt was updated!";

?>
```

php

Si vous utilisez *file_put_contents()* avec le paramètre *FILE_APPEND*, les nouveaux contenus sont ajoutés aux contenus existants.

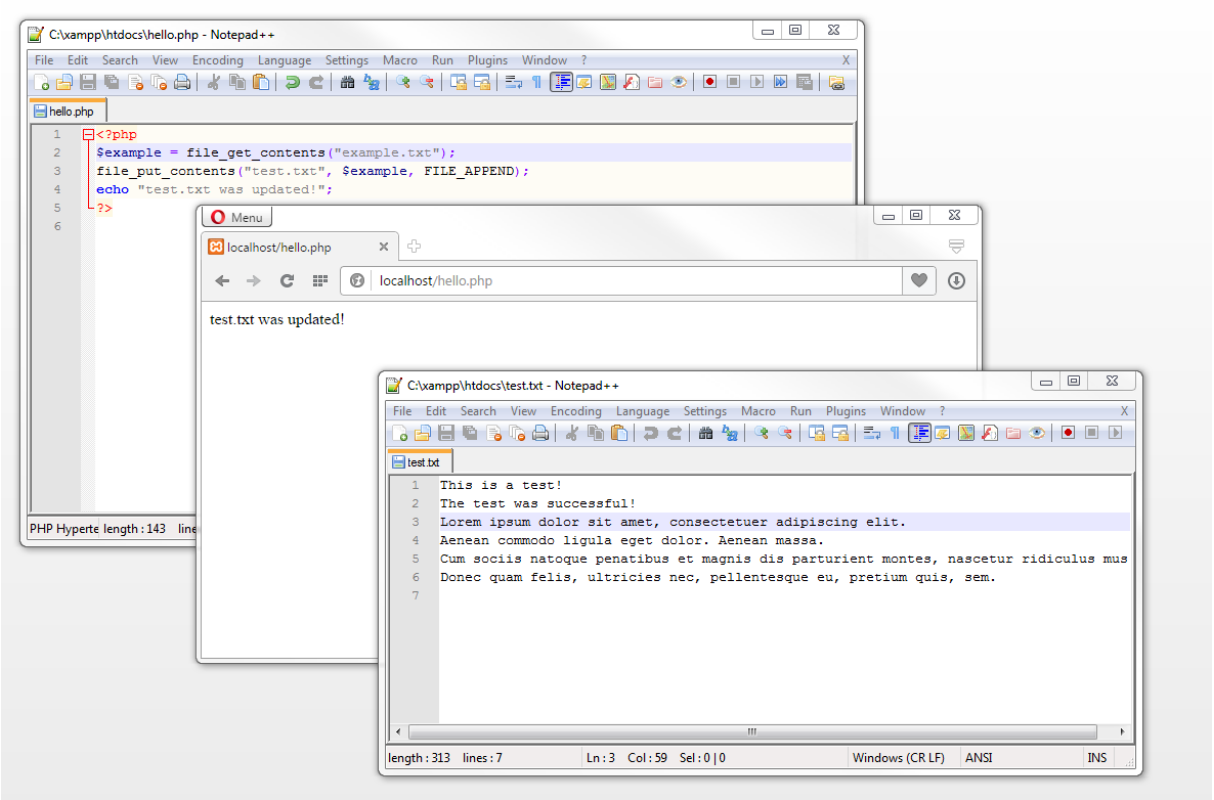


Le script PHP ajoute au fichier test.txt la ligne de texte « The test was successful! ».

Ce que PHP écrit dans le fichier cible n'a pas toujours besoin d'être défini dans le script. Vous pouvez également **transférer du contenu d'un fichier à un autre**. Le script suivant lit le contenu d'example.txt et l'insère dans le fichier test.txt :

```
<?php

$example = file_get_contents("example.txt");

file_put_contents("test.txt", $example, FILE_APPEND);

echo "test.txt was updated!";

?>
```

php



Le script PHP lit les données du fichier example.txt et les intègre dans le fichier test.txt.